



CPE 334

SOFTWARE ENGINEERING · WEEK 8

Architecture, tech stacks and scale

INSTRUCTOR

Dr. Santawat

DEPARTMENT

Computer Engineering, KMUTT

Contents

01

Architecture

what it decides

02

Design and models

structure, behaviour, UML

03

Objects

classes and class diagrams

04

Components and services

the units you assemble

05

Distributed systems

the five patterns

06

Architectural styles

monolith to serverless

07

Choosing an architecture

the selection factors

08

Scaling

one server to many

09

Environments and deployment

where the code runs

10

Technology stacks

MERN and the alternatives

 PART 01

Software Architecture

Software Design and architecture

- Design and documentation take place during the design phase of the SDLC
- Software architecture is the organization of the system
- Serves as a blueprint for developers
- Comprised of fundamental structures and behaviors

— ARCHITECTURE

Software architecture

"Software architecture is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both." — *Len Bass, Paul Clements, and Rick Kazman (Software Engineering Institute / Carnegie Mellon)*

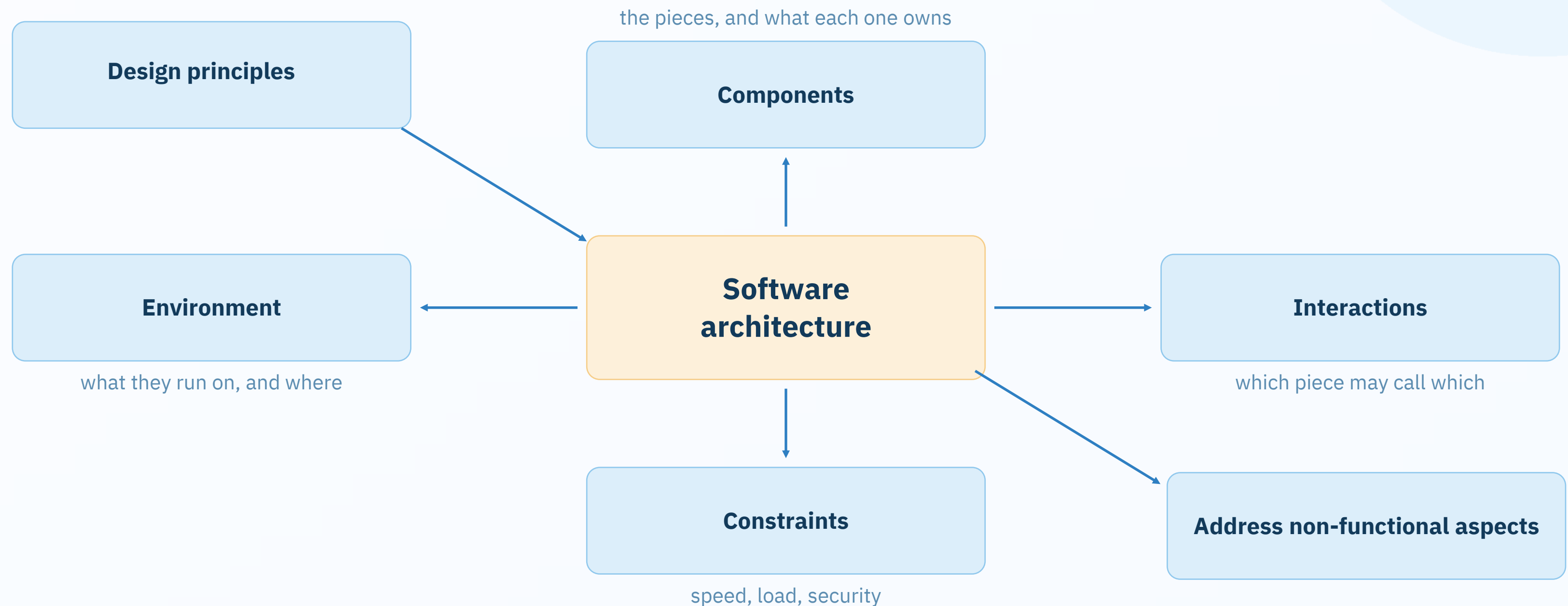
"All architecture is design, but not all design is architecture. Architecture represents the significant design decisions that shape a system, where significant is measured by the *cost of change*."

— **Grady Booch** (Co-creator of UML)

"Architecture is about the important stuff. Whatever that is... [It is] the decisions that you wish you could get right *early in a project, but that you are not necessarily more likely to get right than any other.*"

— **Ralph Johnson & Martin Fowler** (*Patterns of Enterprise Application Architecture*)

Software architecture



Software architecture guide technology stack choice

Why architecture matters



Communication with team

people describe the same system the same way



Earliest design decisions

The expensive choices are made while they are still cheap



Flexibility

Good design allow part to be replaced without touching the rest



Increases lifespan

Good architecture design help software last longer

— ARCHITECTURE

Tech stack

Architecture guides technology stack choice



Language

TypeScript, Python, Go



Framework

Express, Django, Spring



Libraries

the packages you pull in



Data store

PostgreSQL, Redis, S3



Tech stacks must address non-functional capabilities

— ARCHITECTURE

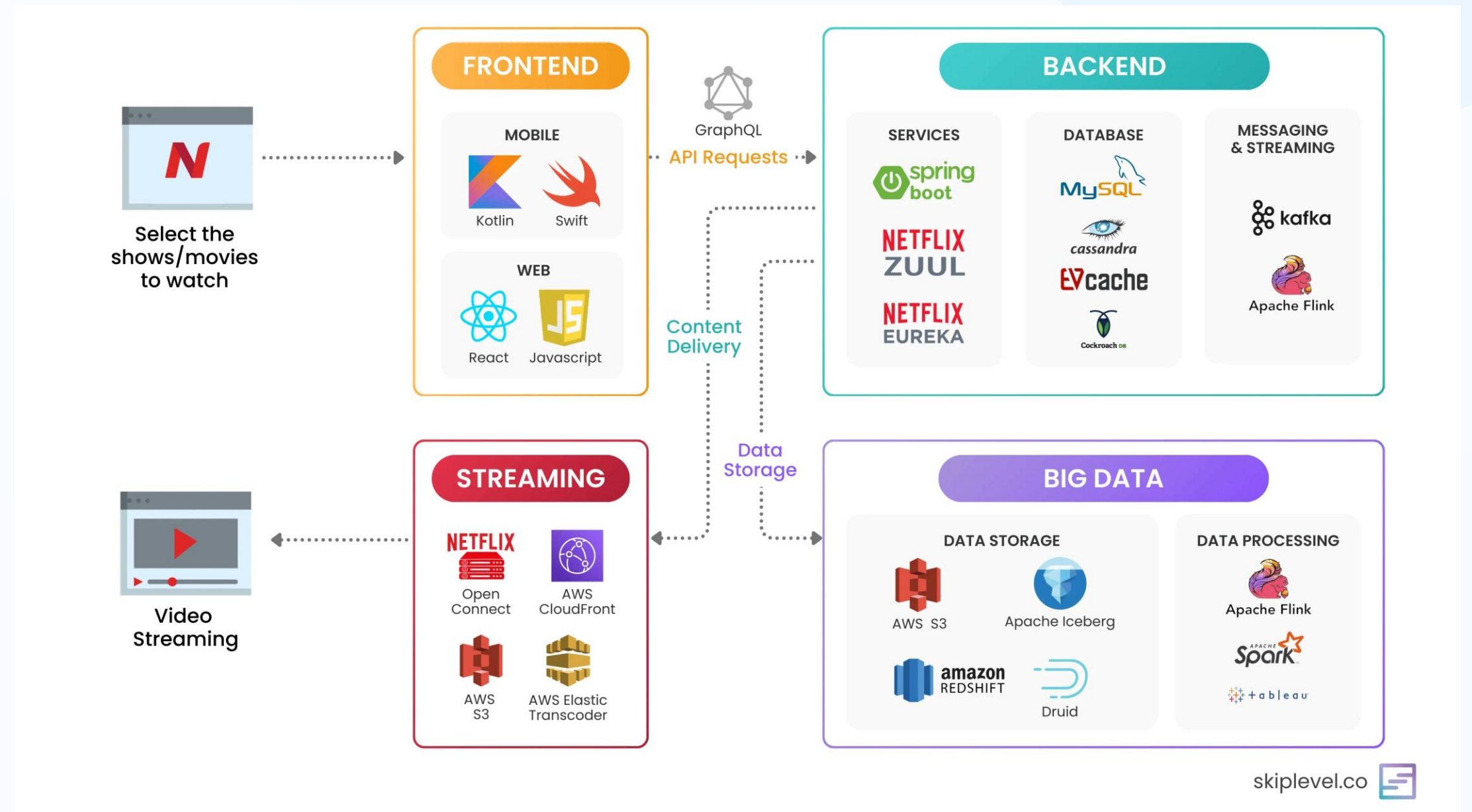
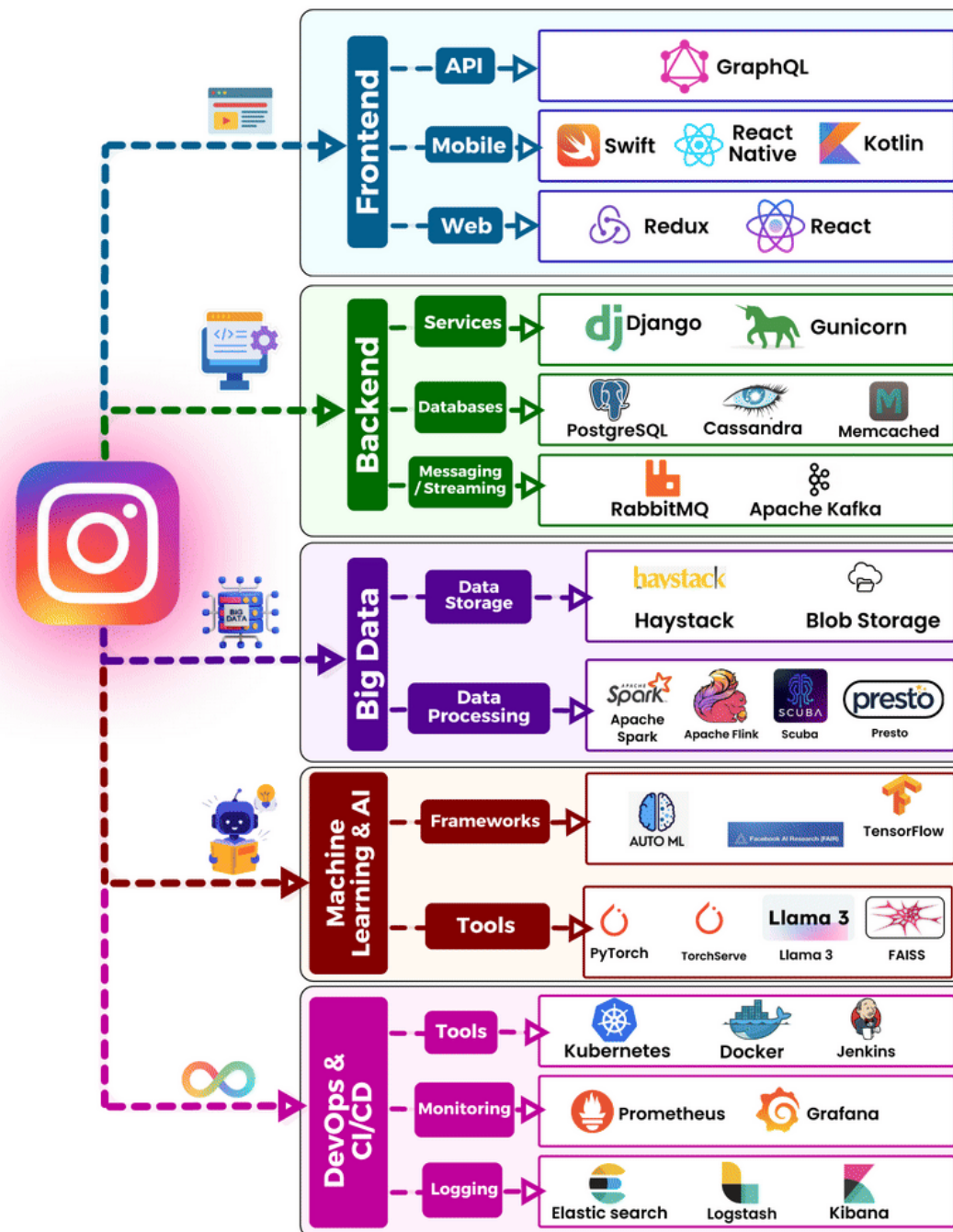


Created by :

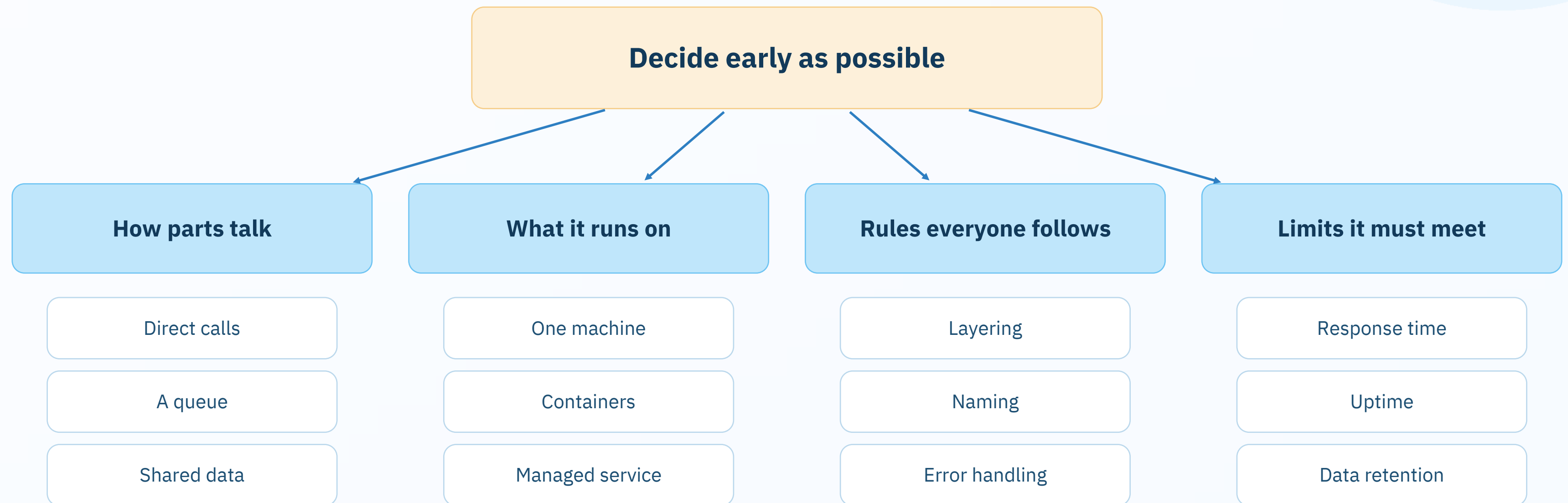
Rocky Bhatia

 courses.learnwithrockybhatia.com
Your Gateway for Tech Learning

Tools and Technologies Used by Instagram



Early design decisions

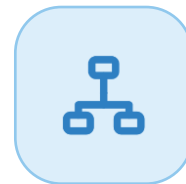


Architecture artifacts



Software Design document (SDD)

Collection of tech specs



Architecture diagram

the parts and the traffic between them



Unified Modeling Language (UML) diagrams

structure and behaviour, in a notation others can read

Software design document

The SDD is a collection of **technical specifications** that indicate how the design should be implemented.

It provides a functional description of the software and design considerations such as

- Assumptions
- Dependencies
- Constraint
- Requirements
- Objectives
- Methodologies

Software design document contents

DESIGN DOCUMENT

01 Introduction and overview

02 System architecture

03 Data design

04 Interface design

05 Component design

06 User interface design

07 Assumptions and dependencies

08 Glossary of terms

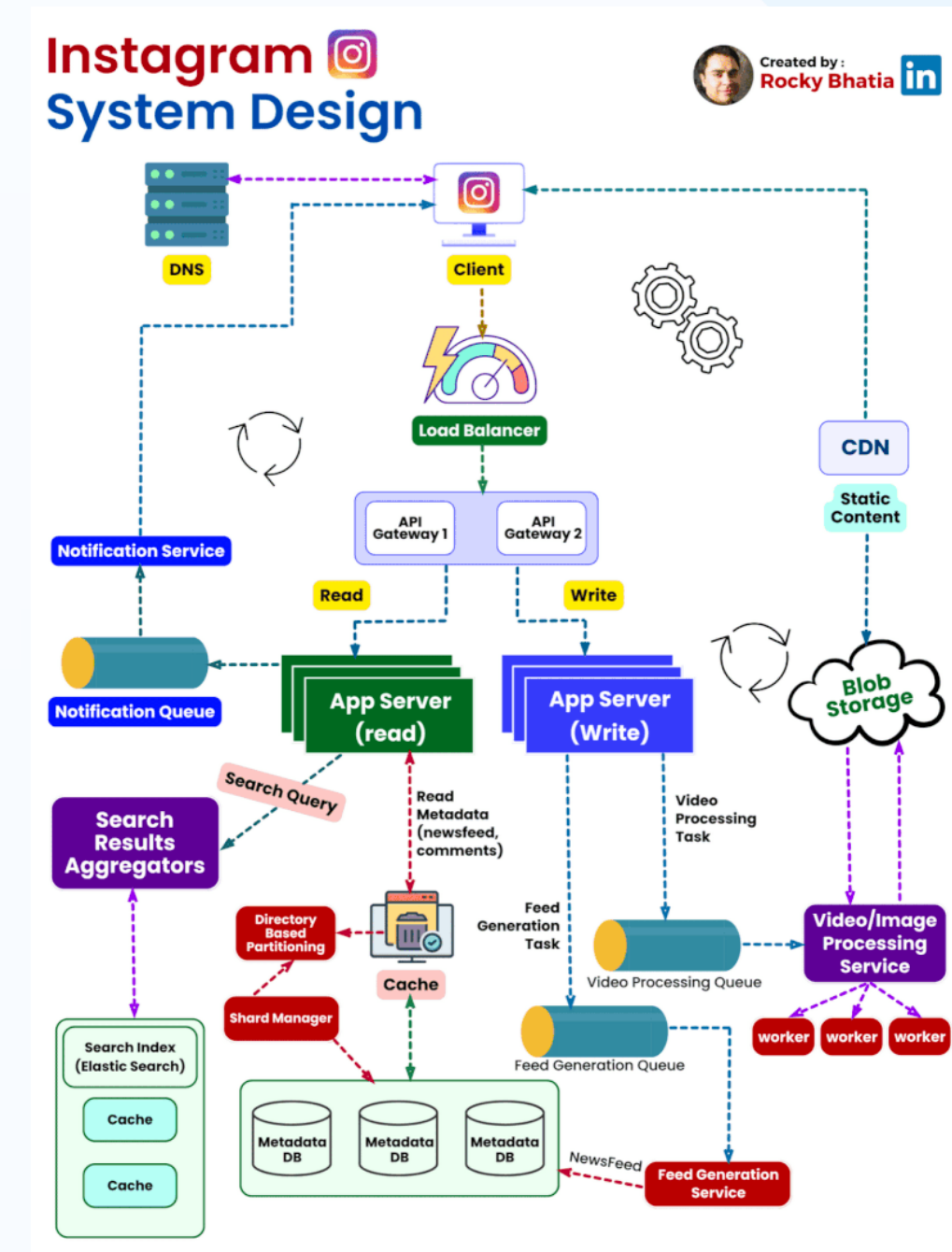
WHAT THE SECTION COVERS

- Goals, project scope and primary features
- The major components and subsystems
- How information is stored, managed and processed
- How the parts communicate with each other
- Detail for each individual module
- How users interact with the system
- What the design takes as given
- Technical terms and acronyms, defined

— ARTIFACTS

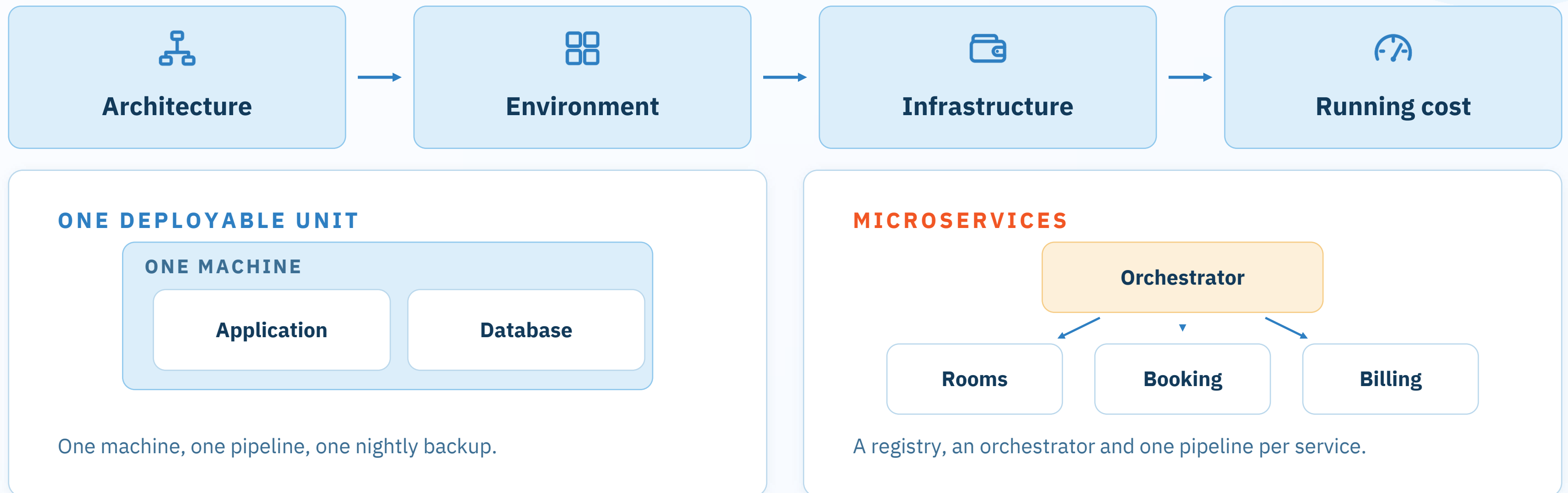
Architecture diagram

- Components
- interactions
- Constraints
- Confines
- Architectural Patterns



— ARCHITECTURE

Architecture changes the deployment



 PART 02

Design and models

Software design

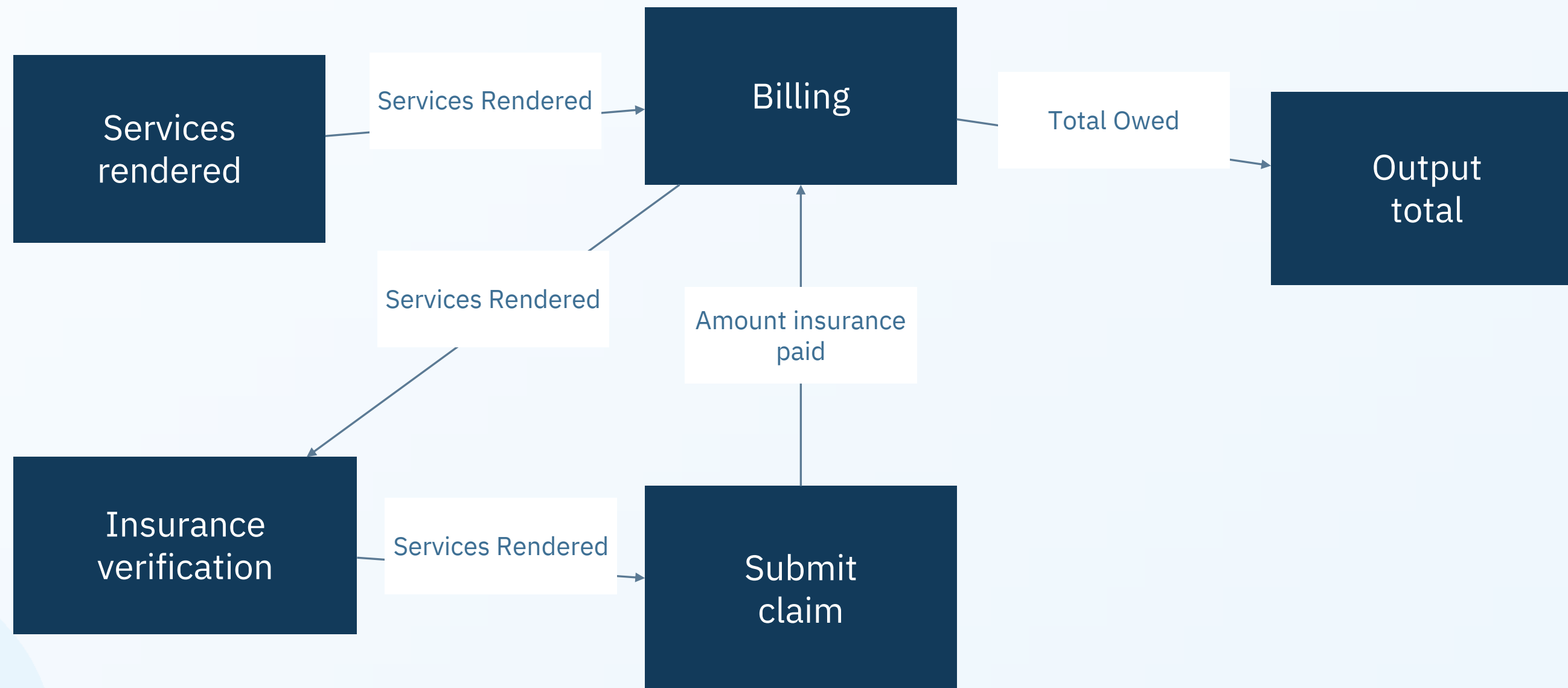
- Software design is a process to document:
 - Structural components
 - Behavioral attributes
- Express software design using:
 - Diagrams and flowcharts
 - Unified Modeling Language (UML)

Characteristics of structured design

- Structural elements: modules & sub-modules
- Cohesive
- Loosely coupled

Structure diagram example

Billing system



Behavioral models

- Describe what a system does but does not explain how it does it
- Communicate the behavior of the system
- Many types of behavioral UML diagrams
 - State transition
 - Interaction

UML Diagrams

- Visual representations to communicate architecture, design, and implementation
- Two types: structural and behavioral
- Programming language agnostic

Advantages of UML

Plan behaviors
and structures
in advance

Facilitates
communication

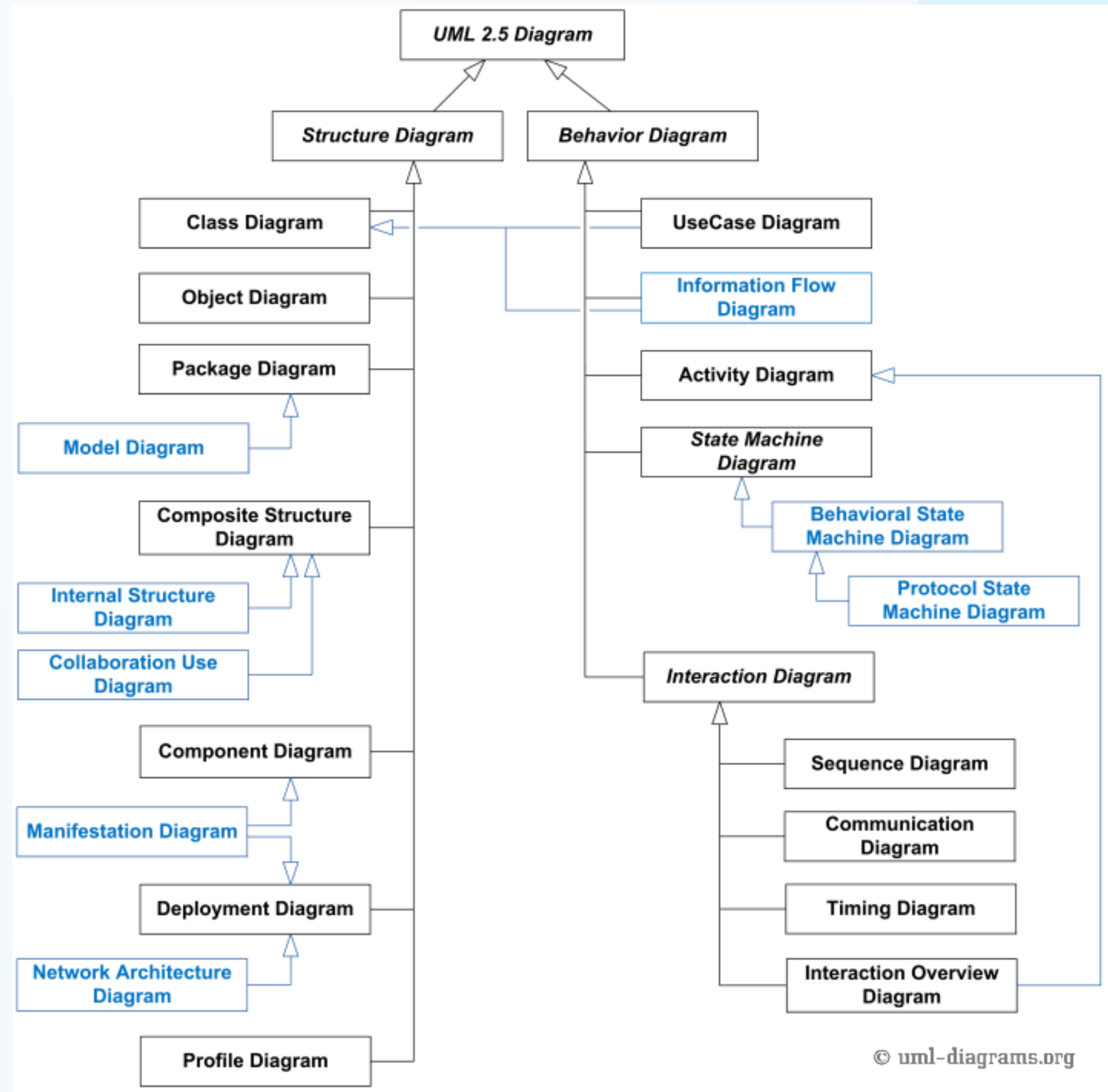
Saves time
and money

Bring team
members up
to speed
quickly

Navigate
source code
easier

Type of UML Diagrams

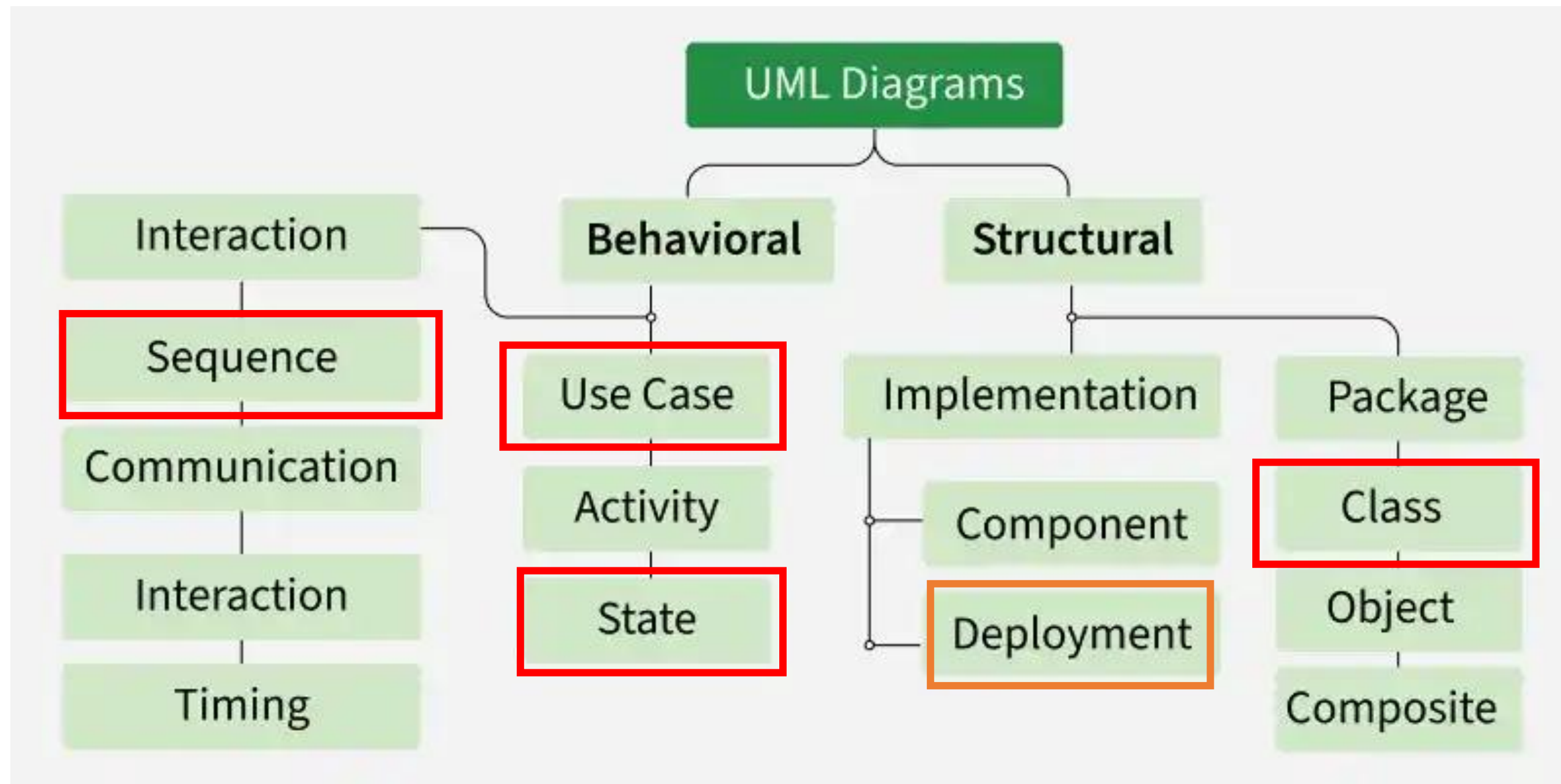
[Unified Modeling Language \(UML\) description, UML diagram examples, tutorials and reference for all types of UML diagrams](#)
- use case diagrams, class, package, component, composite structure diagrams, deployments, activities, interactions, profiles, etc.



UML 2.5 Diagrams Overview.

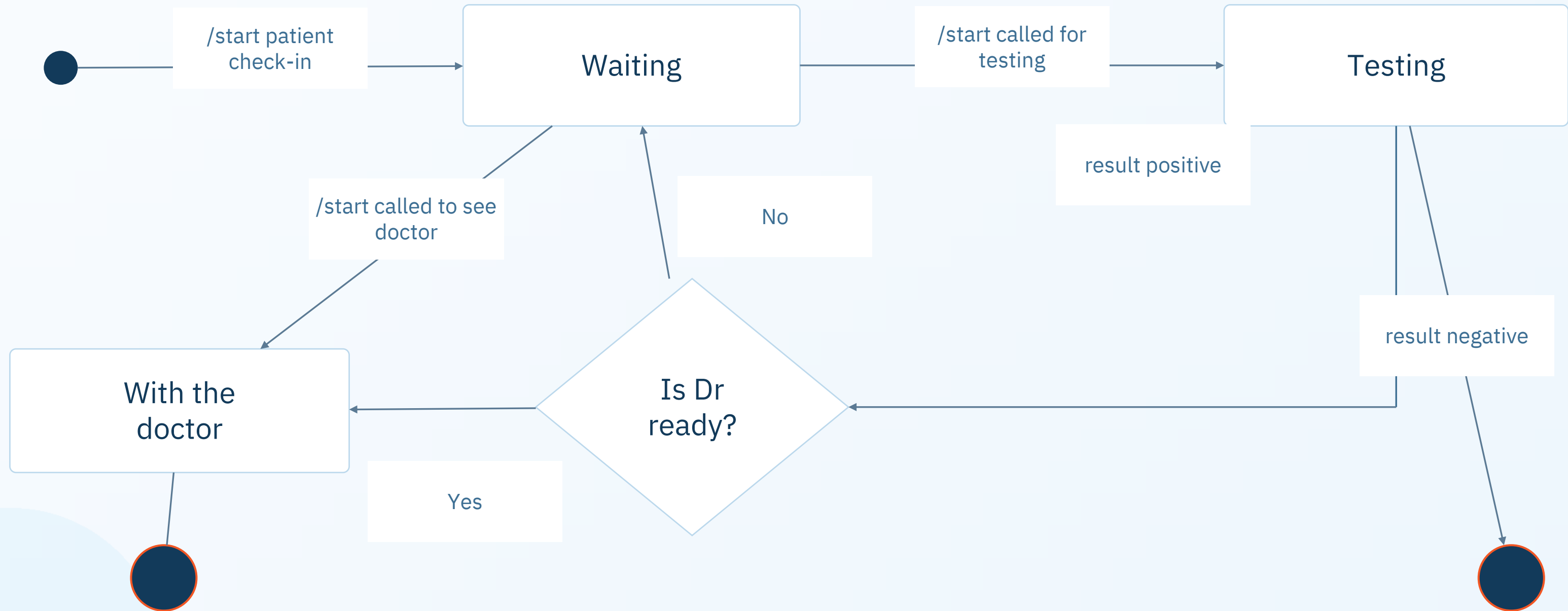
Note, items in blue are not part of official taxonomy of UML 2.5 diagrams

Type of UML Diagrams

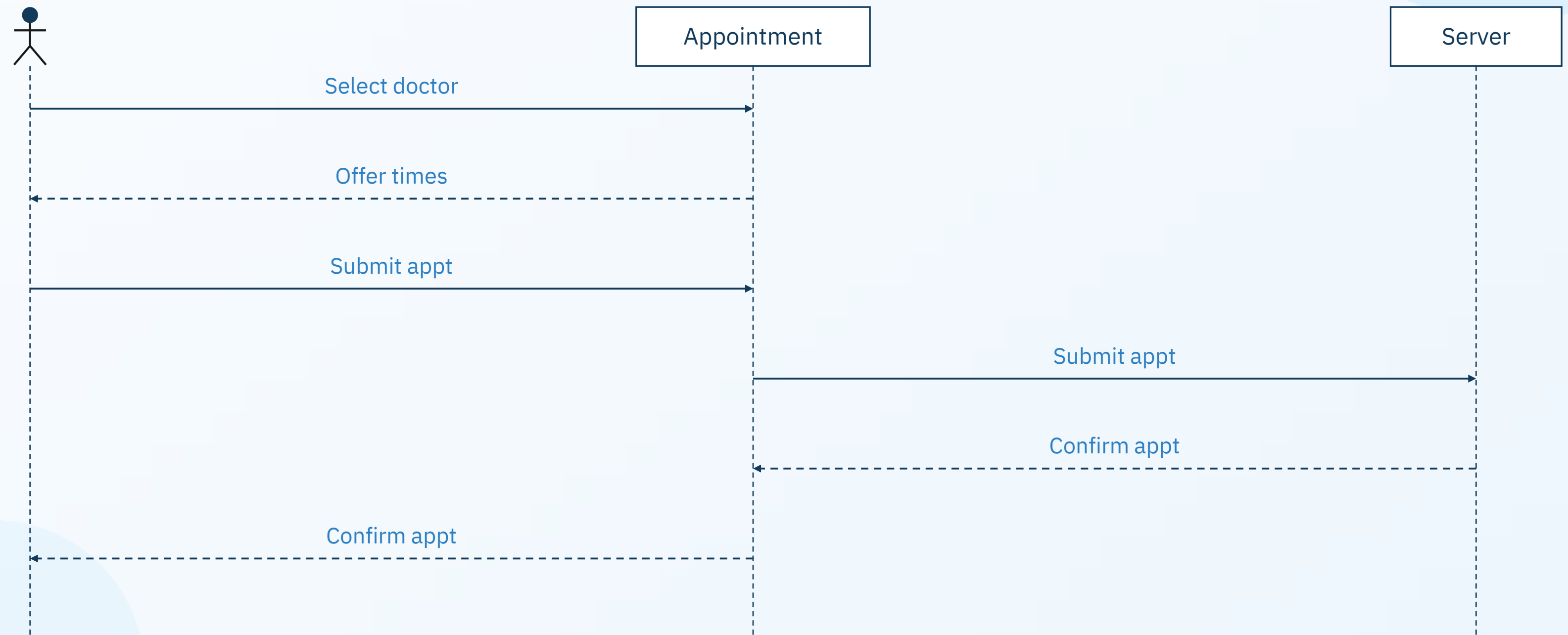


[Unified Modeling Language \(UML\) Diagrams - GeeksforGeeks](https://www.geeksforgeeks.org/unified-modeling-language-uml-diagrams/)

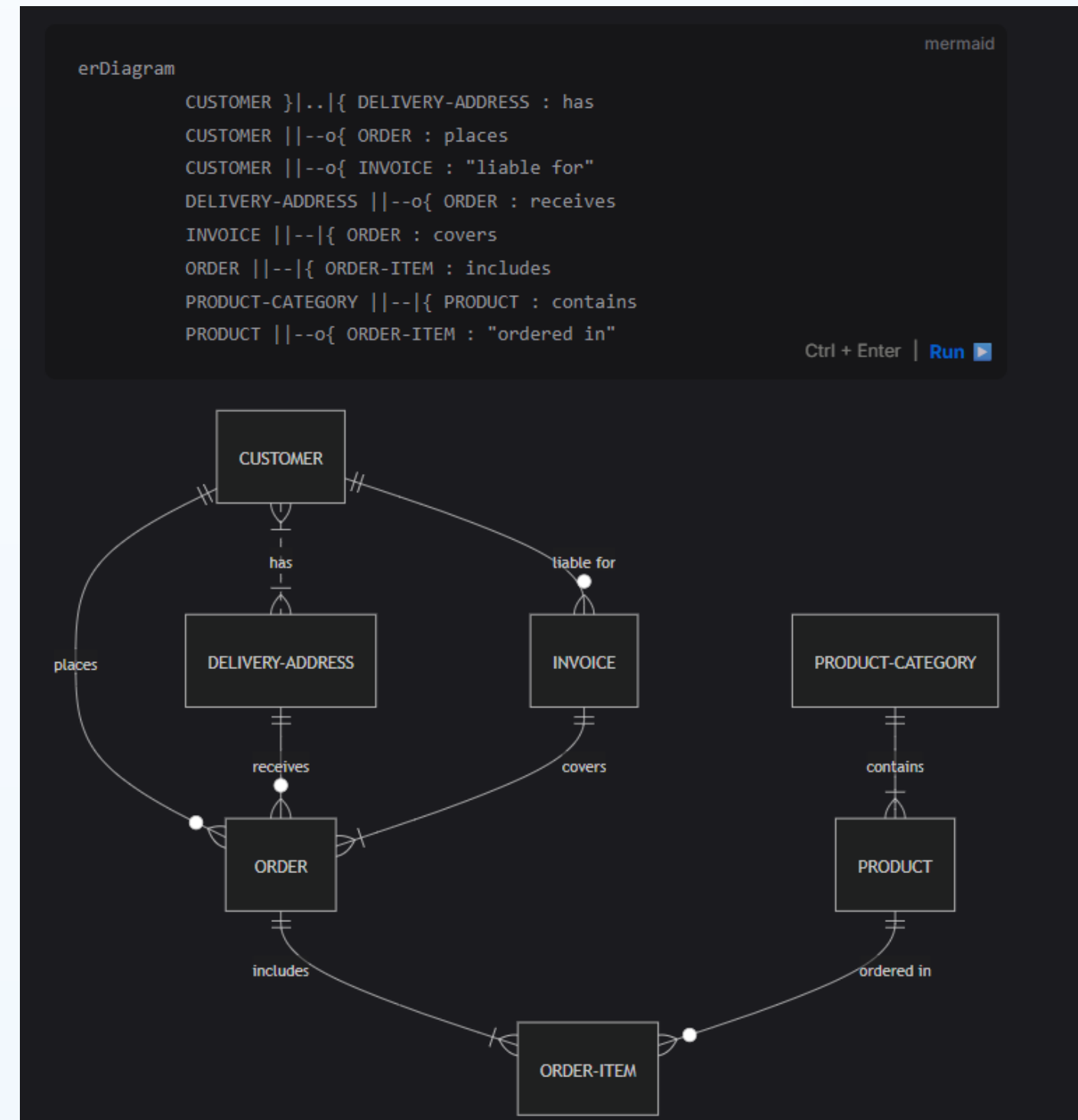
State diagram example



Sequence diagram

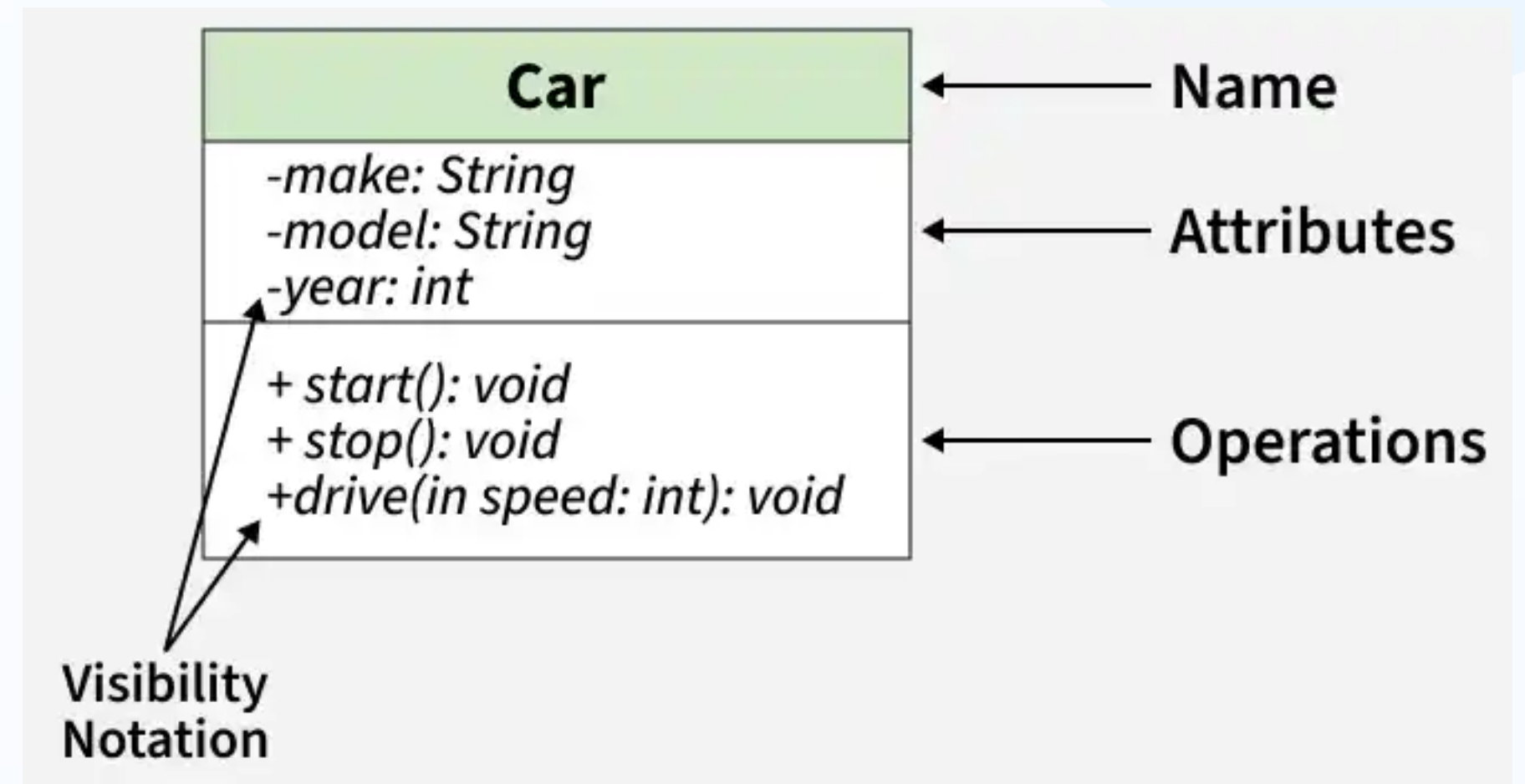


In the age of AI: use mermaid



Classes and objects

- Properties:
 - An object's data
- Methods:
 - An object's actions
 - A class is like a blueprint for an object



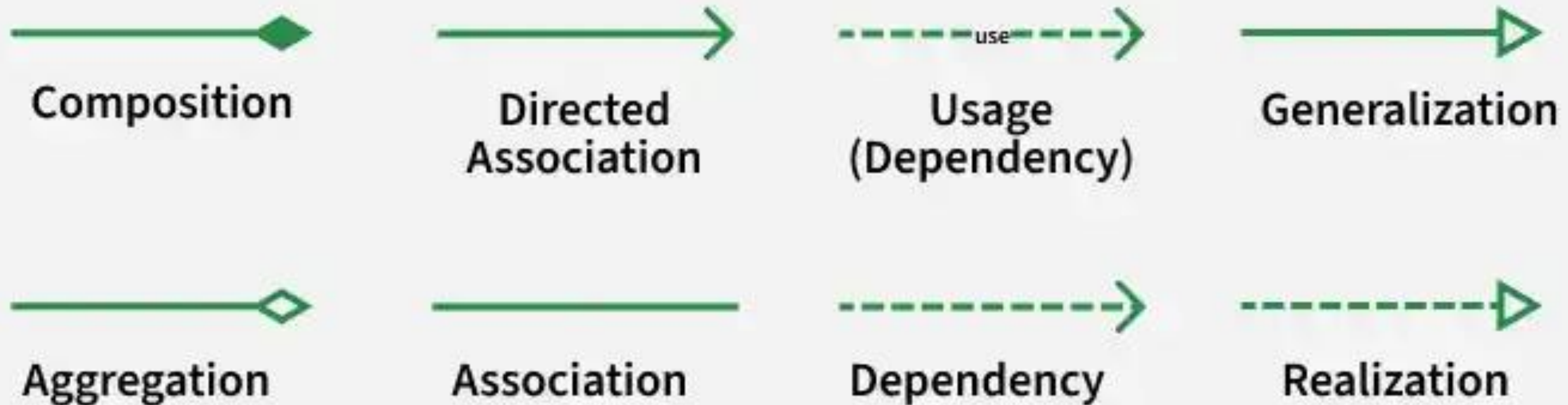
- + for public (visible to all classes)
- - for private (visible only within the class)
- # for protected (visible to subclasses)
- ~ for package or default visibility (visible to classes in the same package)

Object-oriented analysis and design

- Used for a system that can be modeled by interacting objects
- OOAD allows developers to work on different aspects of the same application at the same time
- Visual UML diagrams can be made to show both static structure and dynamic behavior of a system

Object-oriented analysis and design

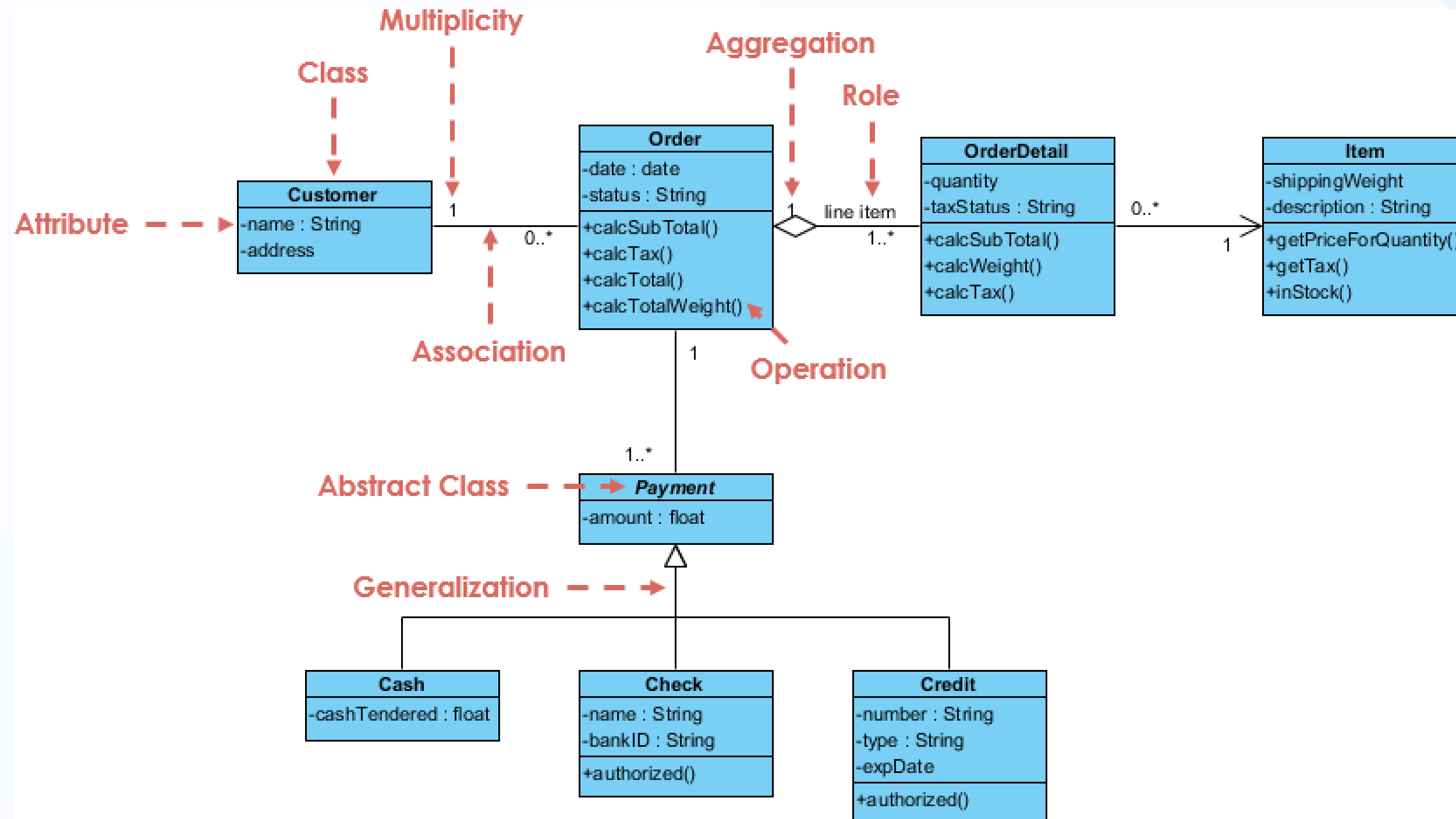
Class Diagram Relationships



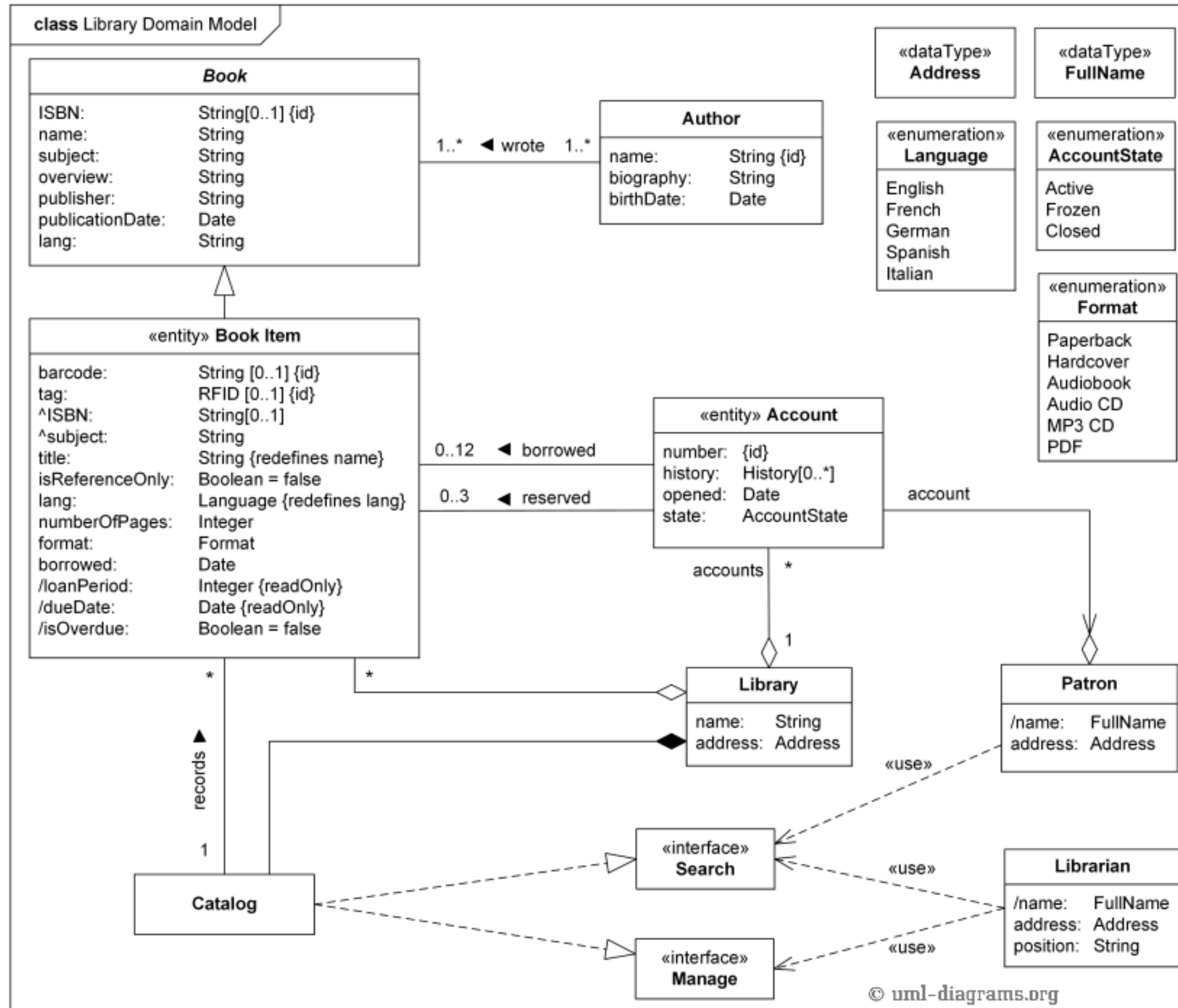
[UML Class Diagram - GeeksforGeeks](#)

Full ref at [UML Class Diagrams - Graphical Notation Reference](#)

Class diagram example



— OBJECTS



[Library domain model UML class diagram example describes main library classes and relationships.](#)

What is a component?

- An individual unit of encapsulated functionality
- Serves as a part of an application in conjunction with other components

Component characteristics

- 1) Reusable
- 2) Replaceable
- 3) Independent
- 4) Extensible
- 5) Encapsulated
- 6) Non-context specific

Component characteristics

- 1) Reusable: reused in different applications
- 2) Replaceable: easily replaced with another component
- 3) Independent: doesn't have dependencies on other components
- 4) Extensible: add behavior without changing other components
- 5) Encapsulated: doesn't expose its specific implementation
- 6) Non-context specific: operates in different environments

Component examples

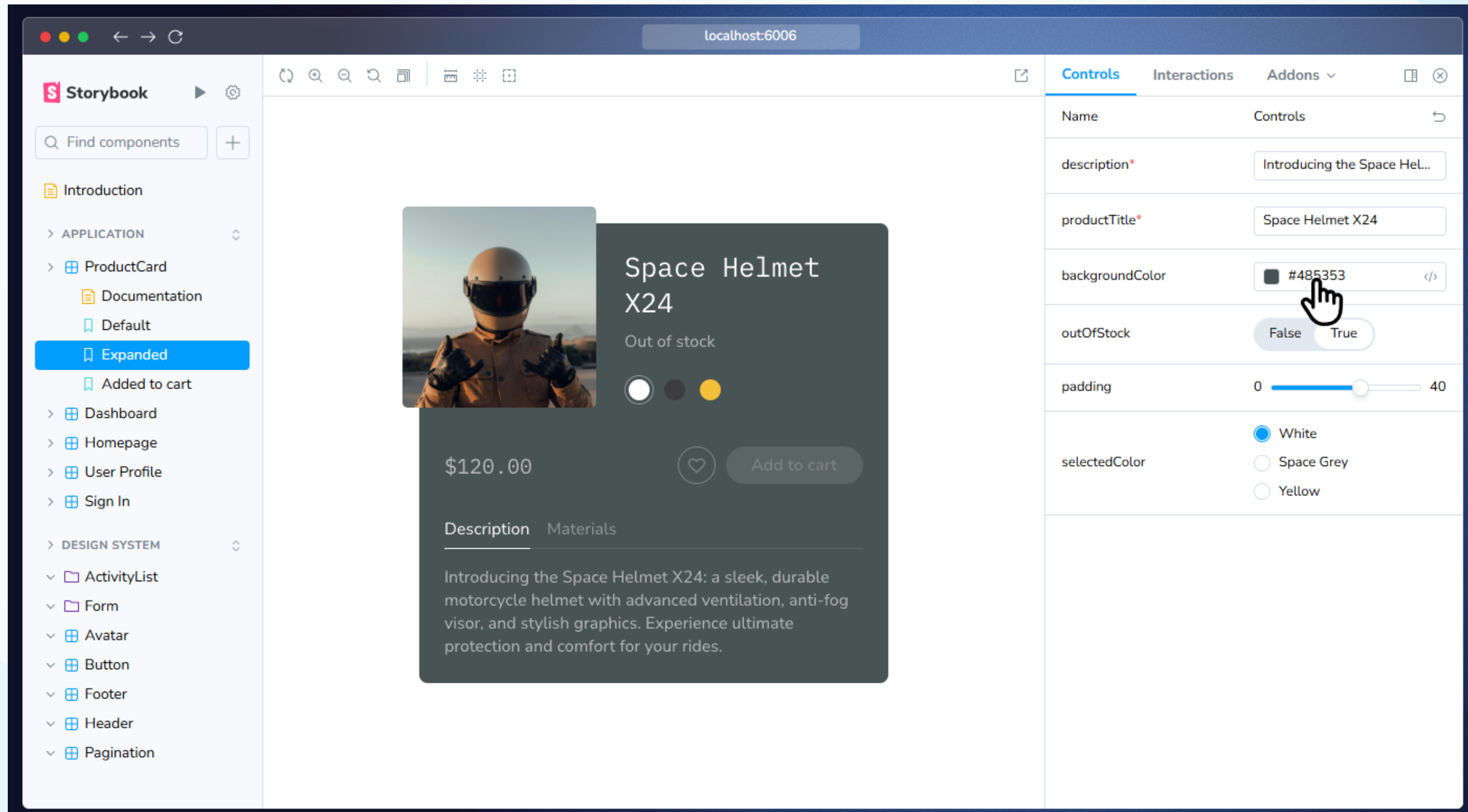
API

Controller

Data Access
Object

— COMPONENTS AND SERVICES

UX/UI design component : story book

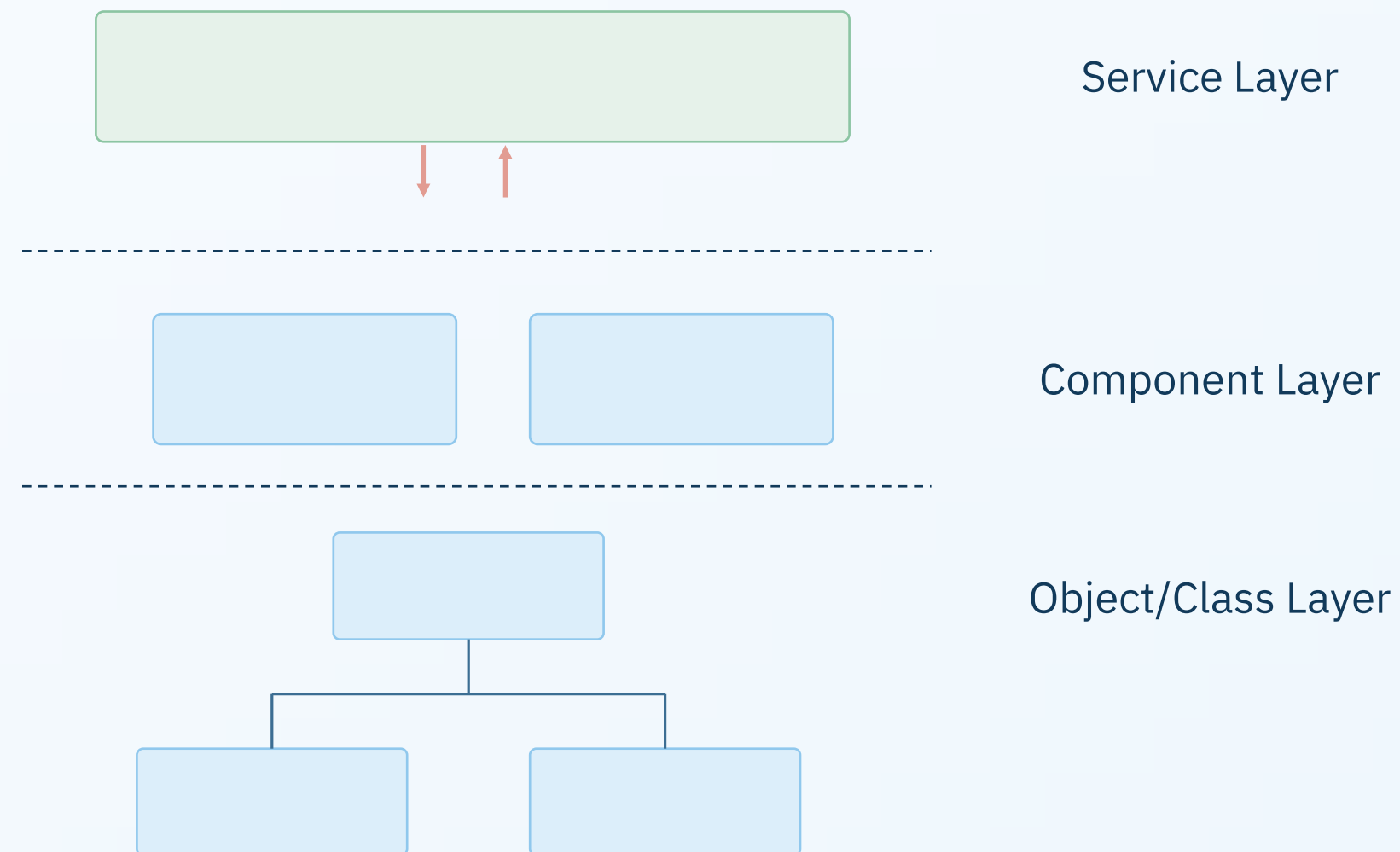


Component-based architecture

- Decomposes design into logical components
- Higher level abstraction than objects
- Defines, composes, and implements loosely coupled independent components so they work together to create an application

Services

- Designed to be deployed independently and reused by multiple systems
- Solution to a business need
- Has one unique, always running instance with whom multiple clients communicate



Examples of Services

- A service is a component that can be deployed independently

Checking a customer's credit

Calculating a monthly loan
payment

Processing a mortgage
application

Distributed systems

- Multiple services located on different machines
- Services coordinate interactions via a communication protocol such as HTTP
- Appears to the end-user as a single coherent system

Distributed system characteristics

- Shares resources
- Fault-tolerant
- Multiple activities run concurrently
- Scalable
- Runs on a variety of computers
- Programmed in a variety of languages

Nodes

- Any device on a network that can recognize, process, and transmit data to other nodes on the network
- Distributed systems have multiple interconnected nodes running services

 PART 06

Architectural styles

Types of architectural styles

In this lecture

2-tier

3-tier

Peer-to-peer

Event-driven

SOA

Microservices

Serverless

Architectural pattern
for you to check out

Model-view-
controller

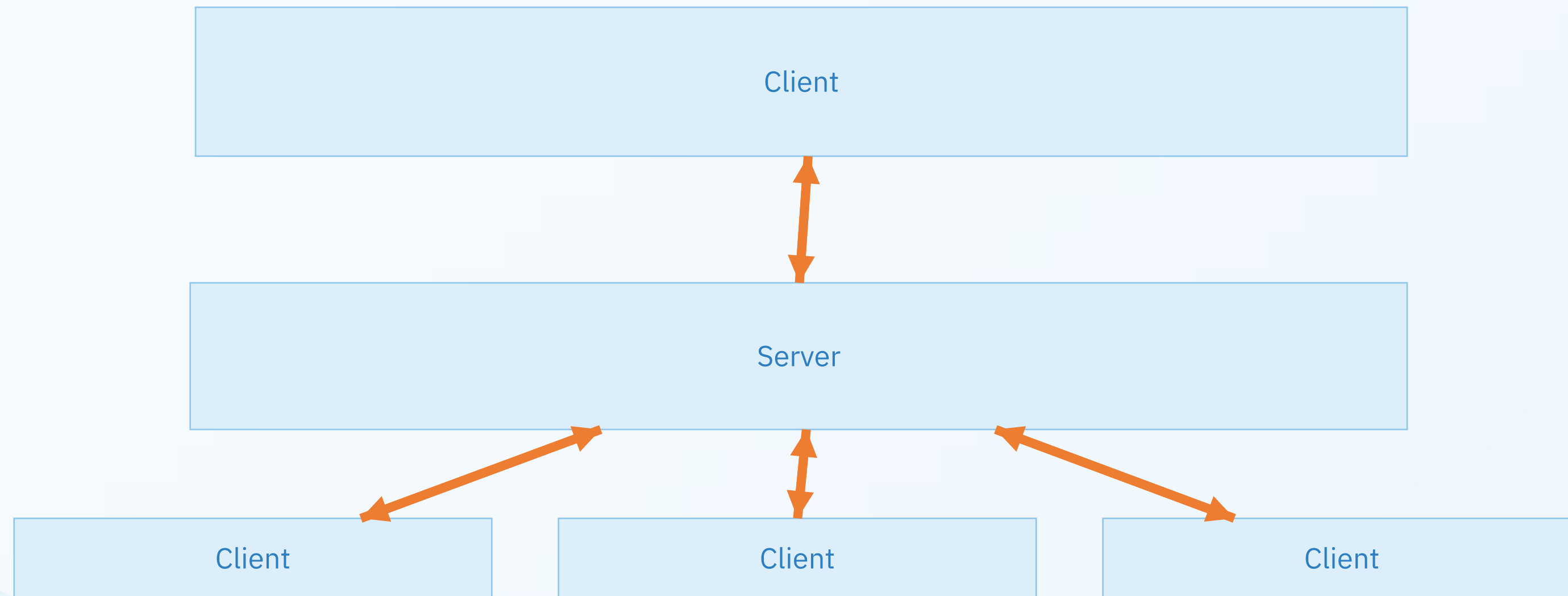
Message-
broker

Blackboard

Pipe-filter

Controller-
responder

2-tier



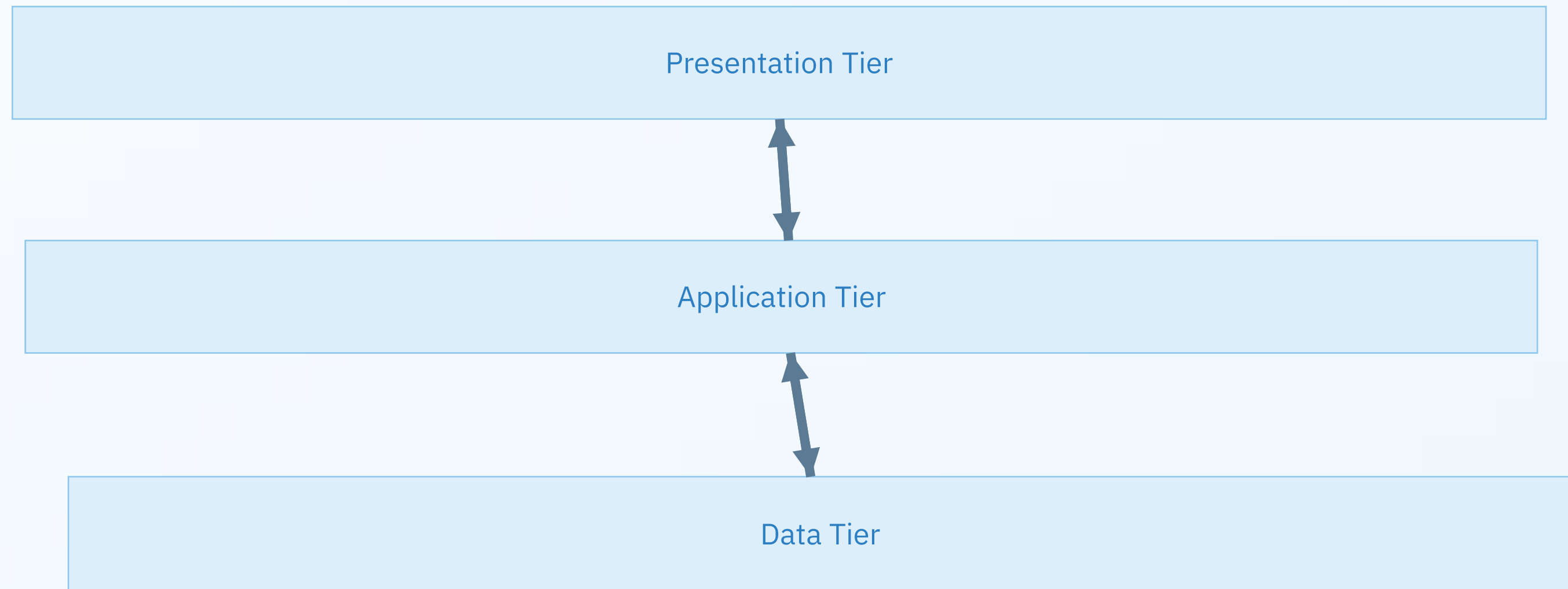
IN INDUSTRY



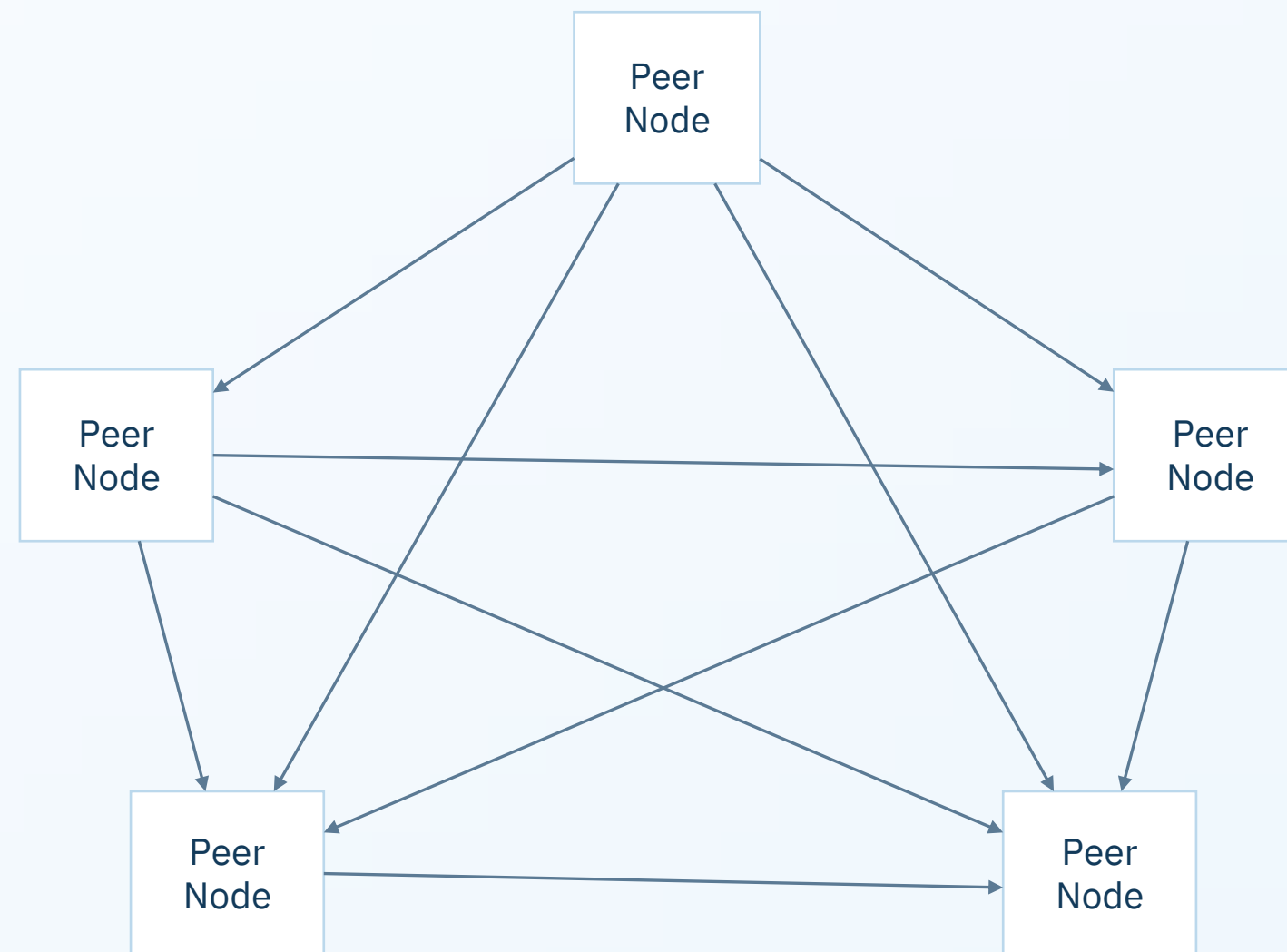
Small desktop and mobile apps that talk straight to a managed database — Firebase, Supabase, an in-house Access or SQL Server tool — the client is the application tier.

3-tier

Or n tier



Peer-to-peer (P2P)

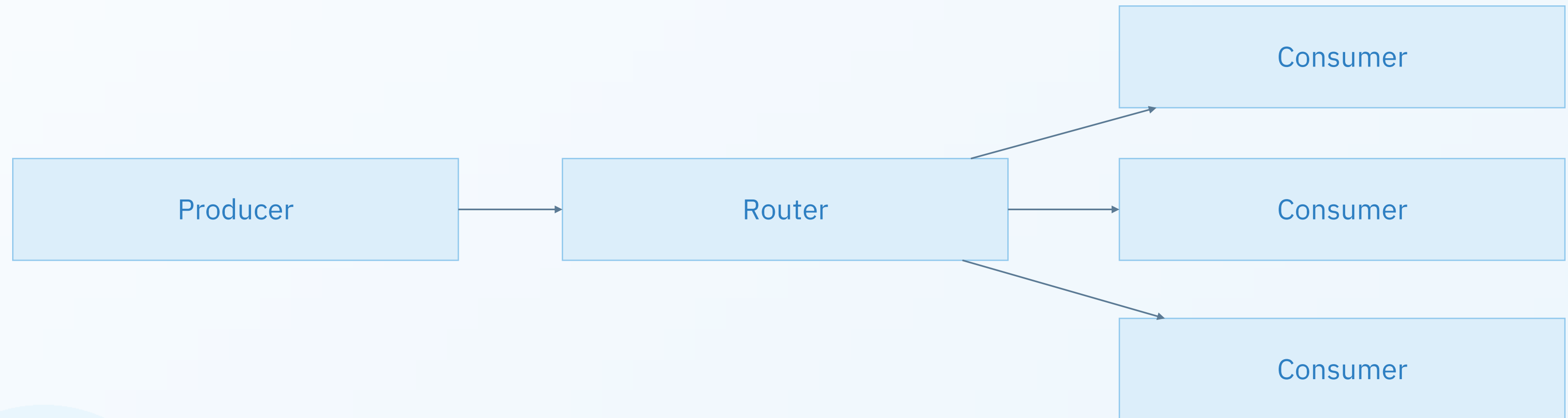


IN INDUSTRY



BitTorrent for file distribution, Bitcoin — “a peer-to-peer electronic cash system” — and IPFS for content-addressed storage.

Event-driven



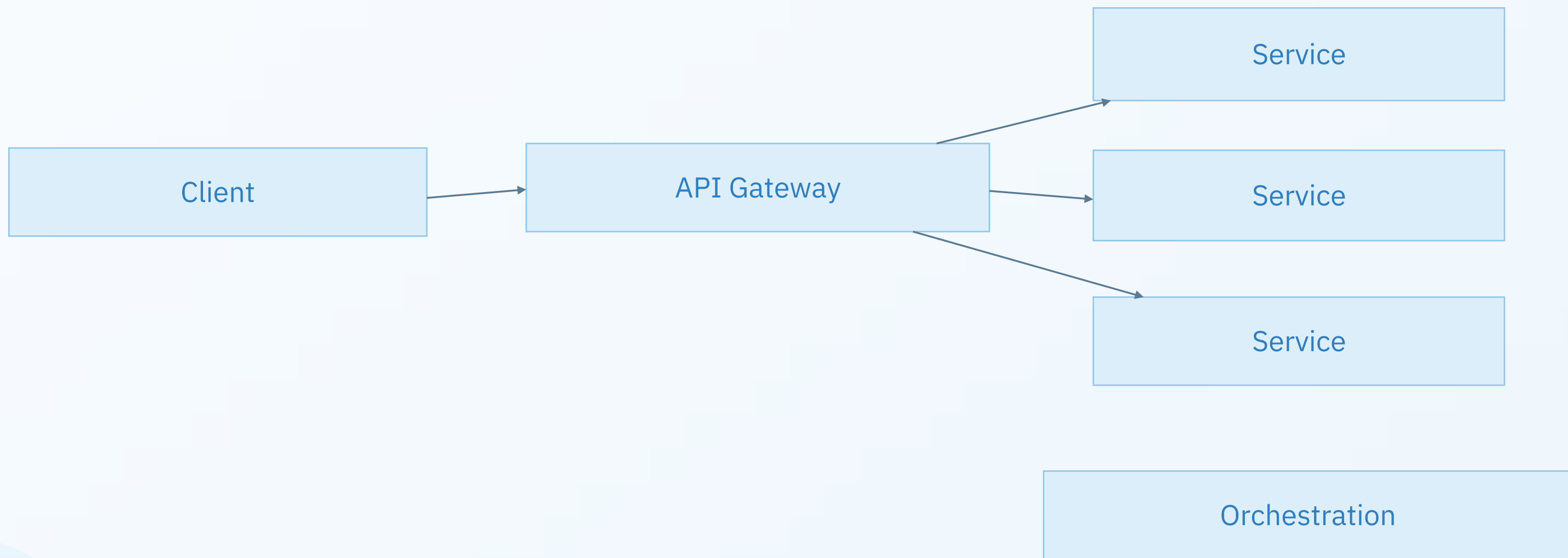
IN INDUSTRY

Uber



Uber runs Apache Kafka as the pub/sub bus between its rider and driver apps — trillions of messages and multiple petabytes a day.

Microservices



IN INDUSTRY



Netflix puts Zuul, an L7 gateway, in front of its backend services; Amazon and Uber are built the same way.

Examples

- 2-tier: Messaging apps - Amazon, Gmail, Dropbox, Moodle
- 3-tier: Web apps - CRM, e-commerce, banking apps, WordPress
- Event-driven: Ride sharing, stock market analysis, smart-home IoT, fraud detection, multiplayer games, chatbots.
- Peer-to-peer: Cryptocurrency, BitTorrent, Bitcoin, Ethereum
- Microservices: Amazon, eBay, Netflix, Spotify, banking, health records, Twitter, Facebook

Combining styles

- Some patterns can be combined in a single system
- Example:
 - 3-tier with microservices
 - Peer-to-peer with event-driven
- Some patterns cannot be combined
- Example:
 - Peer-to-peer with two-tier



Monolith VS SOA VS Microservice VS Serveless

What is Monolith?

- Has all or most of its functionality within a single process
- Managed in internal layers for:
 - Security
 - Reporting
 - Analytics
 - User interface
 - Data access
- Interconnected
- Interdependent

IN INDUSTRY



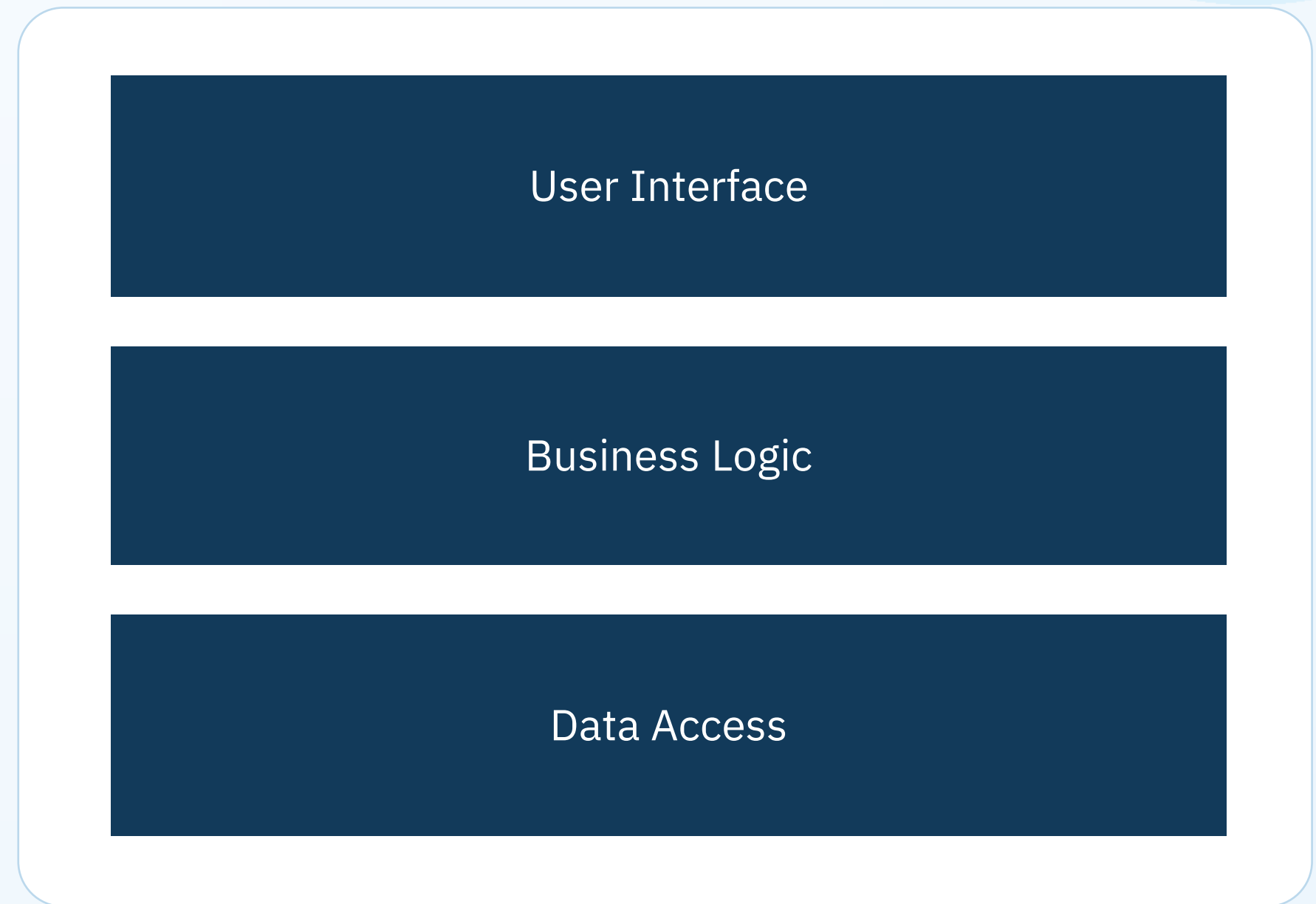
Shopify's core commerce app is a monolith — 2.8 million lines of Ruby and 500,000 commits — componentised in place rather than split into services.

Benefits and drawbacks of Monolith

- Benefits:
 - Simple
 - Less cross-cutting
- Drawbacks:
 - With growth, complexity increases
 - Less flexibility to adapt to new technology

Monolith - Example

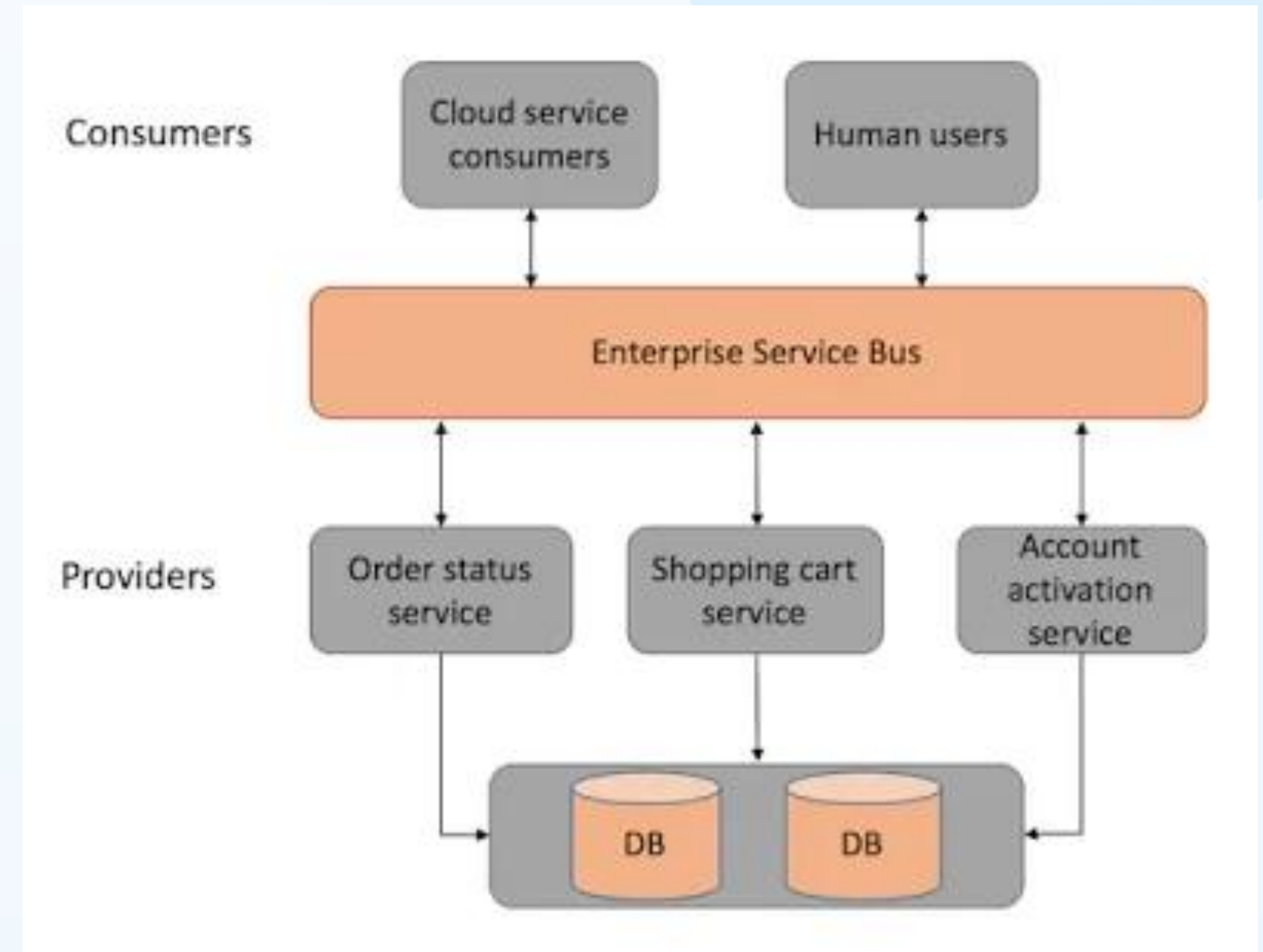
- Windows Forms Application



— ARCHITECTURAL STYLES

What is SOA?

- Designed and built in a service provider and consumer approach
- Provides services that are:
 - Seamless
 - Reusable
- Comprises:
 - Interface
 - Contract
 - Implementation



<https://www.javacodegeeks.com/2016/11/microservices-vs-soa.html>

IN INDUSTRY

Amazon's 2002 internal mandate — every team exposes its data and function only through service interfaces — is the most reported SOA turn in industry.

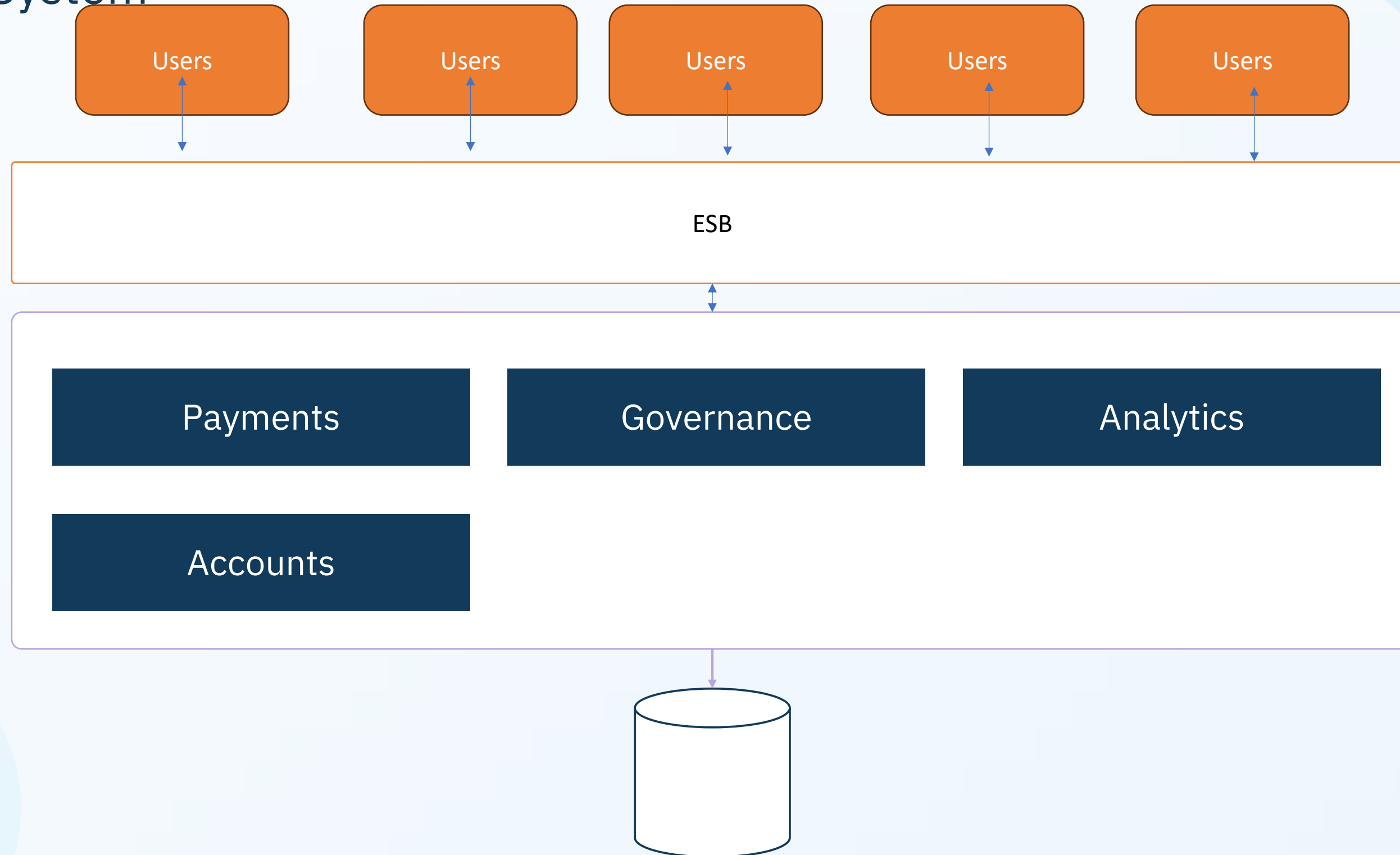
Benefits and drawbacks of SOA

- Benefits:
 - Increases reliability
 - Supports parallel development
- Drawbacks:
 - Complex design
 - Expensive investment

— ARCHITECTURAL STYLES

SOA - Example

- Banking System



What are Microservices?

An approach to create a single application composed of:

- Many loosely coupled
- Independently deployable
- Smaller services
- Microservice data is not shared
 - Scalable
 - Technology independent

IN INDUSTRY



SoundCloud moved off a Rails monolith to independent services, each owned by one team.

Benefits and drawback of microservices

Benefits:

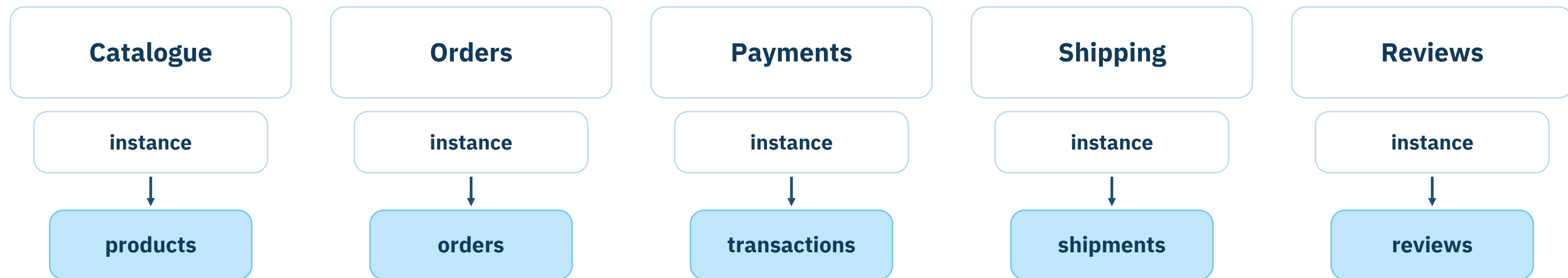
- No interdependency = Easy updates
- Flexibility to add new technology
- Smaller components = Scale independently

Drawbacks

- Security requirement
- Debugging is a challenge

Microservices: an e-commerce example

EACH SERVICE OWNS ITS OWN DATA



They exchange events; none of them reads another service's store

Scaling microservices

- Horizontal scaling
- Precise scaling
- Event driven

— ARCHITECTURAL STYLES

Scaling microservices

BLACK FRIDAY • ONLY THE BUSY SERVICE GROWS

Catalogue

instance

Checkout

instance

instance

instance

Search

instance

instance

Billing

instance

Four services, eight instances — chosen per service, not per system

WHAT DECIDES THE INSTANCE COUNT



Kubernetes



Docker



AWS Auto Scaling

Patterns for microservices

- Enable some of the more common challenges and opportunities
- Examples:
 - Single-page application (SPA)
 - Backend for Frontend (BFF)
 - Strangler
 - Service discovery

Single page application (SPA)

- Enabled by more powerful browsers, faster networks, and client-side languages
- Loads one interface that never reloads
- Updates dynamically using calls to backend REST-based services
- Simplifies the front-end experience
- Places greater performance responsibility on backend services

IN INDUSTRY

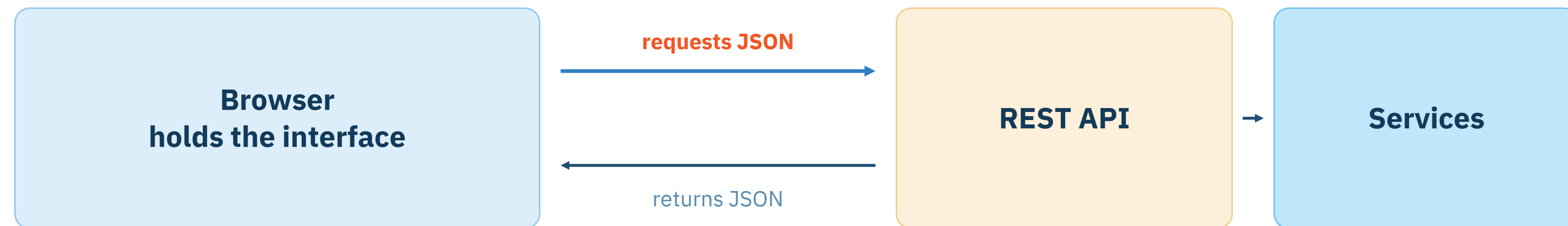


Gmail, Figma and Trello — one page that never reloads, updating through calls to backend APIs.

— ARCHITECTURAL STYLES

Single page application

ONE DOCUMENT LOAD • MANY DATA CALLS



The page is never reloaded — only the data changes

FRONT-END FRAMEWORKS THAT BUILD ONE



React



Angular



Vue.js



Next.js

Backend for Frontend (BFF)

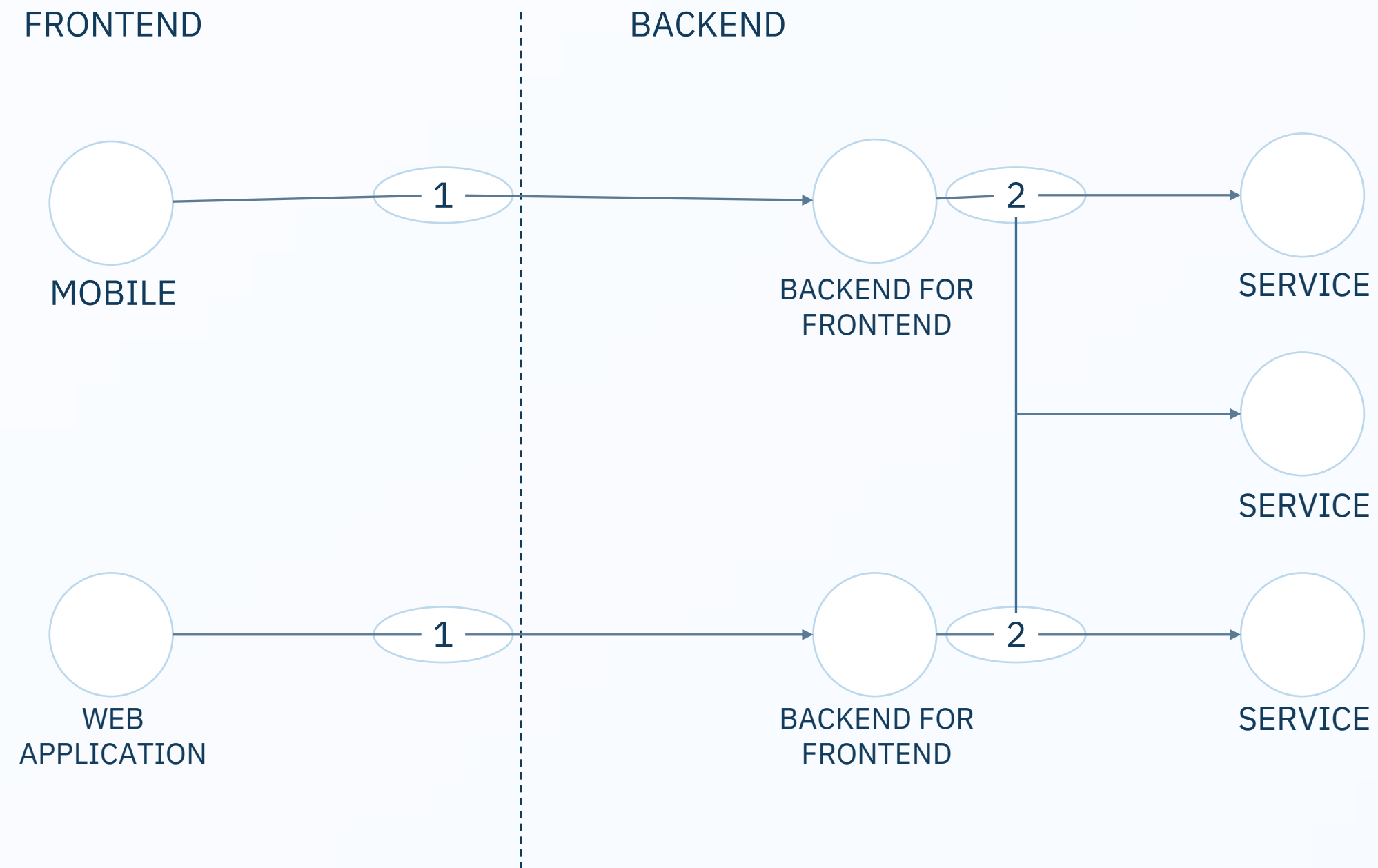
- Provides superior support compared to a generic backend
- Inserts a layer between user experience and the resources
- Enables customized user experiences for different channels
- Supports one backend type per user interface

IN INDUSTRY



The pattern comes from SoundCloud: one backend each for the web, iOS and Android front ends.

Backend for Frontend (BFF)



Strangler pattern

- Manages refactoring in stages
- Gets its name from a vine that strangles a tree
- Splits functional domains
- Replaces with new microservice per domain
- Enables two applications to exist side-by-side

IN INDUSTRY



Shopify strangled its own monolith in place — domains carved out module by module while the old application kept serving traffic.

Strangler pattern

Transform	Coexist	Eliminate
Build a new site beside the existing one	Leave the existing site running	Remove the replaced functionality
Implement one functional domain at a time	Redirect each new function to the new site	Delete it once traffic no longer reaches it

A router decides, per route, which system answers

Service discovery

- Helps applications and services discover each other
- Provides flexibility as service instances can change dynamically
- Could be used by load balancers to perform health checks and rebalance traffic on service failures

IN INDUSTRY

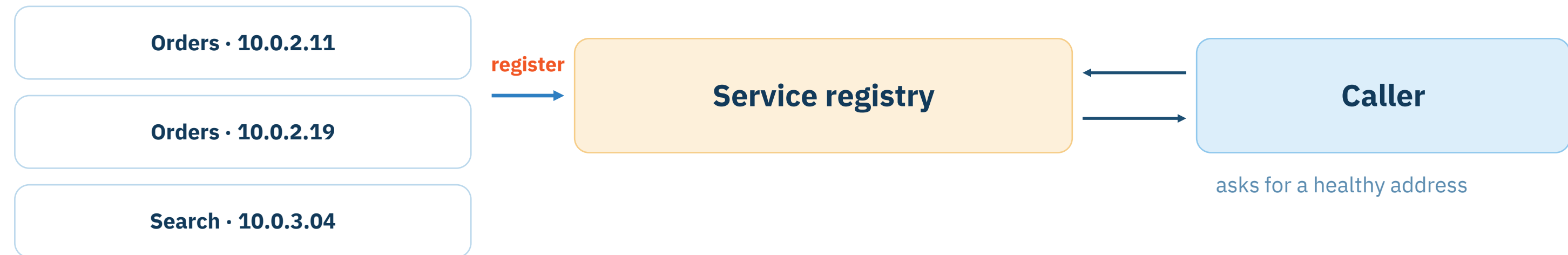


Netflix Eureka, HashiCorp Consul and Kubernetes' built-in DNS discovery all do this job in production.

— ARCHITECTURAL STYLES

Service discovery

INSTANCES COME AND GO · ADDRESSES ARE NOT FIXED
RUNNING INSTANCES



WHAT PROVIDES DISCOVERY IN PRACTICE



Kubernetes



Docker



NGINX



Envoy

Other patterns

- Entity and aggregate patterns
 - Example: e-commerce site
- Adapter patterns
 - Help translate relationships between incompatible objects
 - Example: integration with third-party API

Microservice don'ts

- 1 Don't start with microservices**

Refactor a monolith into services when its complexity starts to hurt development and maintenance — not before.
- 2 Don't skip automation**

Deployment and monitoring must be automated, or run on managed cloud services, before the infrastructure sprawls.
- 3 Don't build nanoservices**

Split too far and the coordination between the pieces costs more than the split ever returns.
- 4 Don't drift back into SOA**

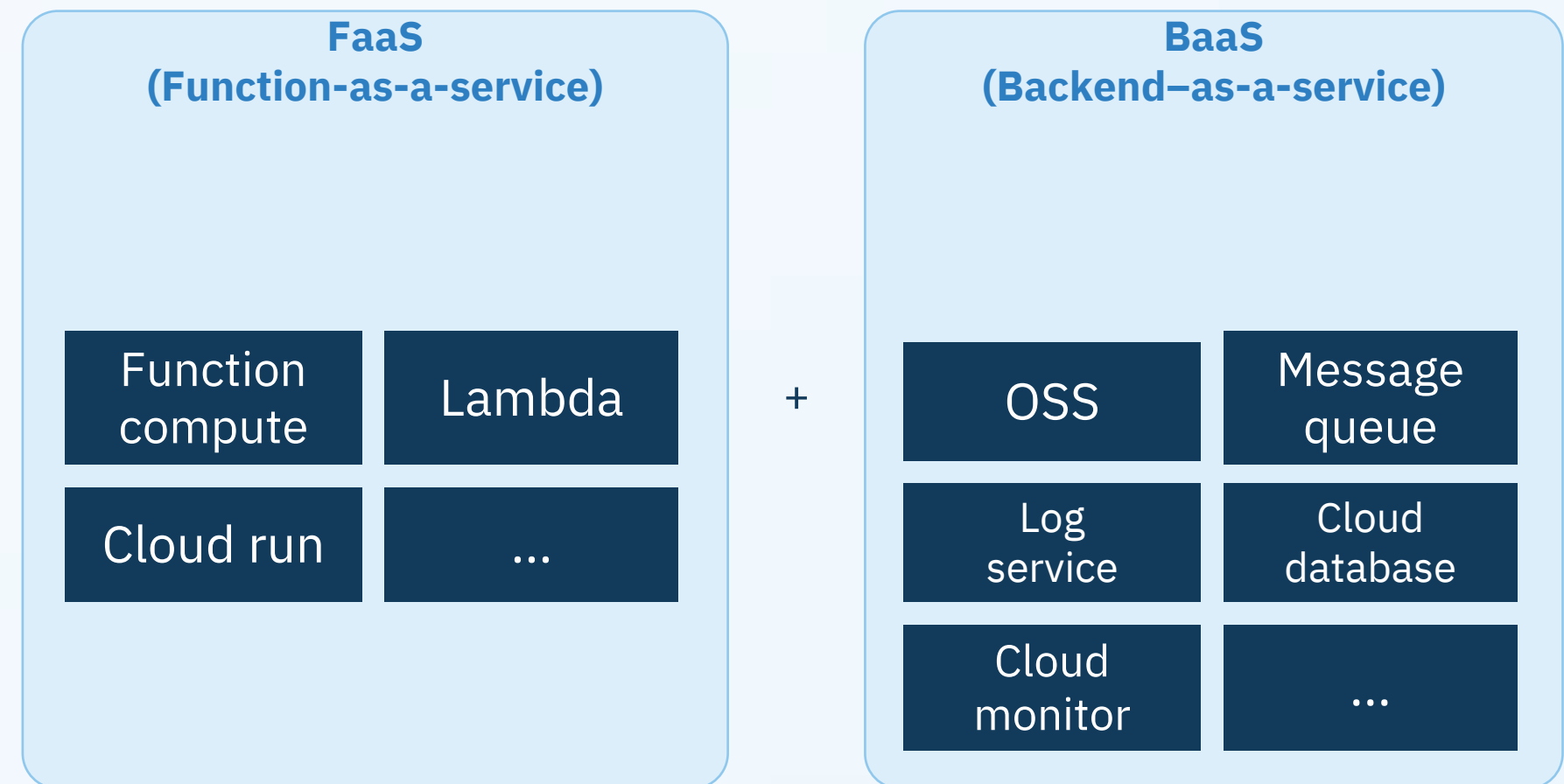
Microservices stay fine-grained, each owning its own data store — that is the bounded context.
- 5 Don't give every service its own gateway**

One API gateway sits between the client and the services, and carries the cross-cutting concerns once.

— SERVERLESS

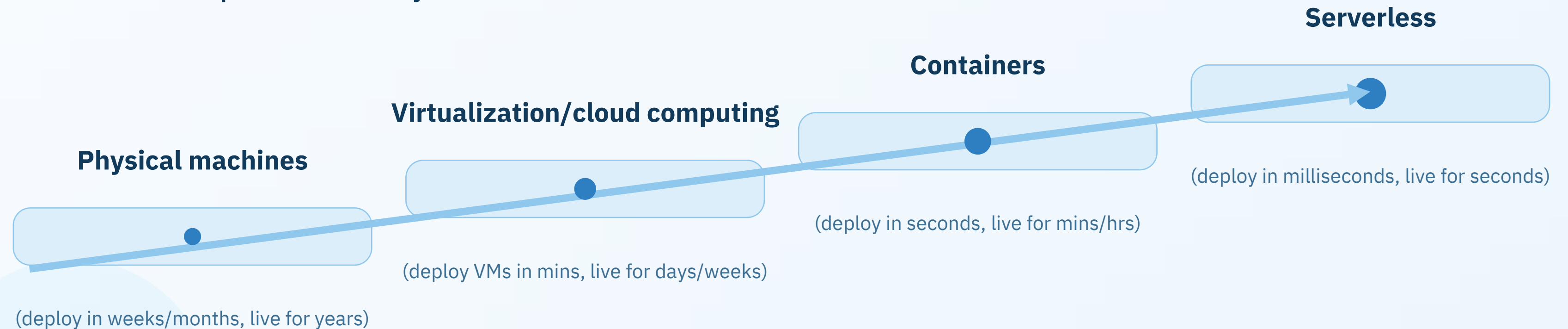
Serverless = FaaS + BaaS

- Think of serverless computing as a combination of FaaS platforms and BaaS services
- FaaS platforms are used to run functions
- BaaS represents backend cloud services, such as databases, object storage, and message queues



Evolution of serverless computing

- Progression from traditional to serverless
- Faster deployments
- Shorter lifespans
- Increased productivity



Serverless architecture concepts

- Abstracts infrastructure and software environment
- Code runs in a cloud platform
- Cloud provider manages the hardware and software setup, security, performance
- Billed only for usage
- Developers only need to focus on applications and code in the form of functions

IN INDUSTRY

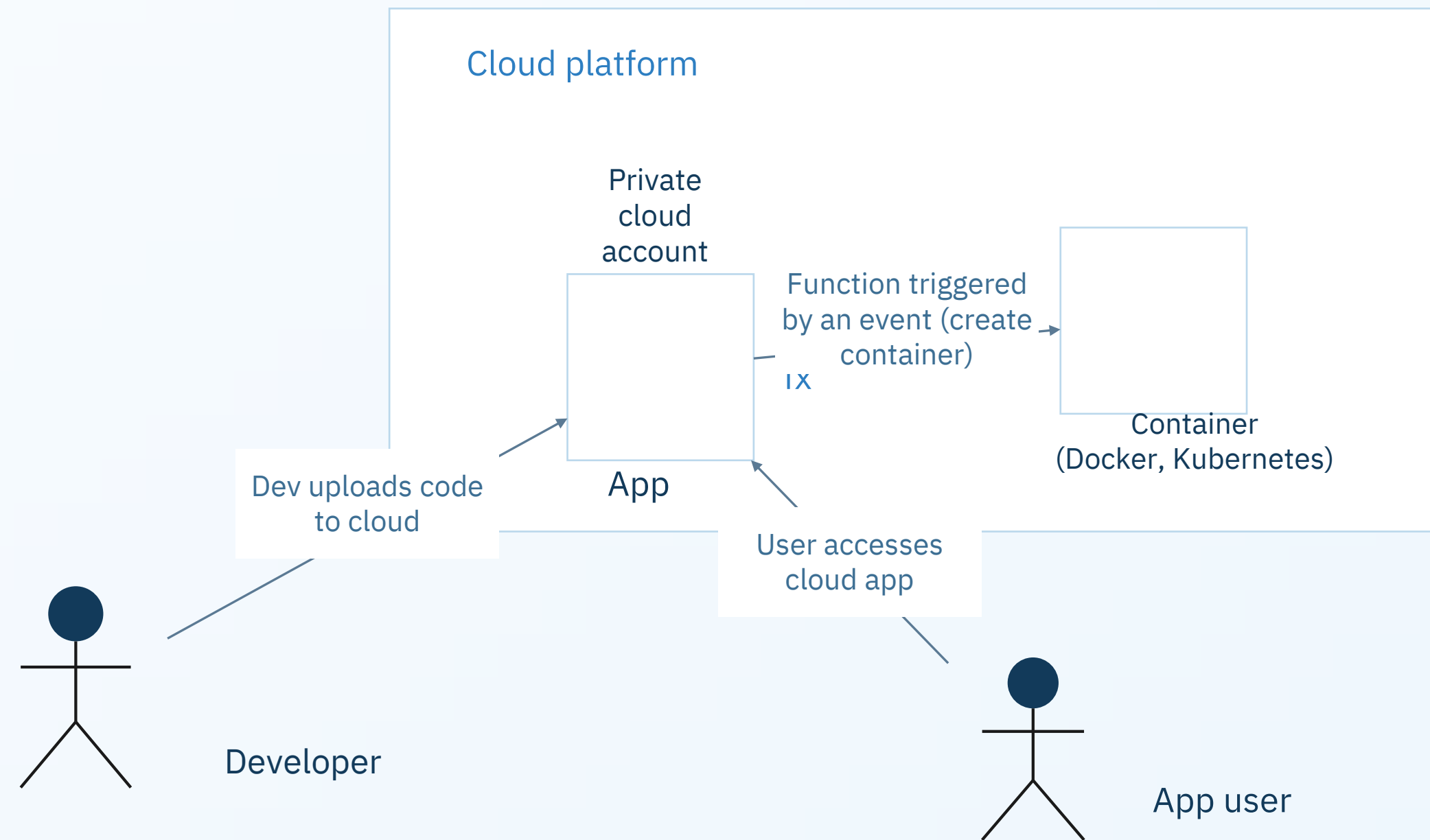


FINRA validates 37 billion market events a day on Lambda; Toyota Connected handles 18 billion transactions a month; iRobot runs its IoT backend this way.

Serverless characteristics

- Hostless
- Elastic
- Load balanced
- Stateless
- Event driven
- Highly available
- Usage-based/granular billing

How serverless functions work



Serverless: benefits and constraints

BENEFITS

No server to provision or patch

Scales automatically with demand

Idle cost approaches zero

Fast from empty account to running endpoint

CONSTRAINTS

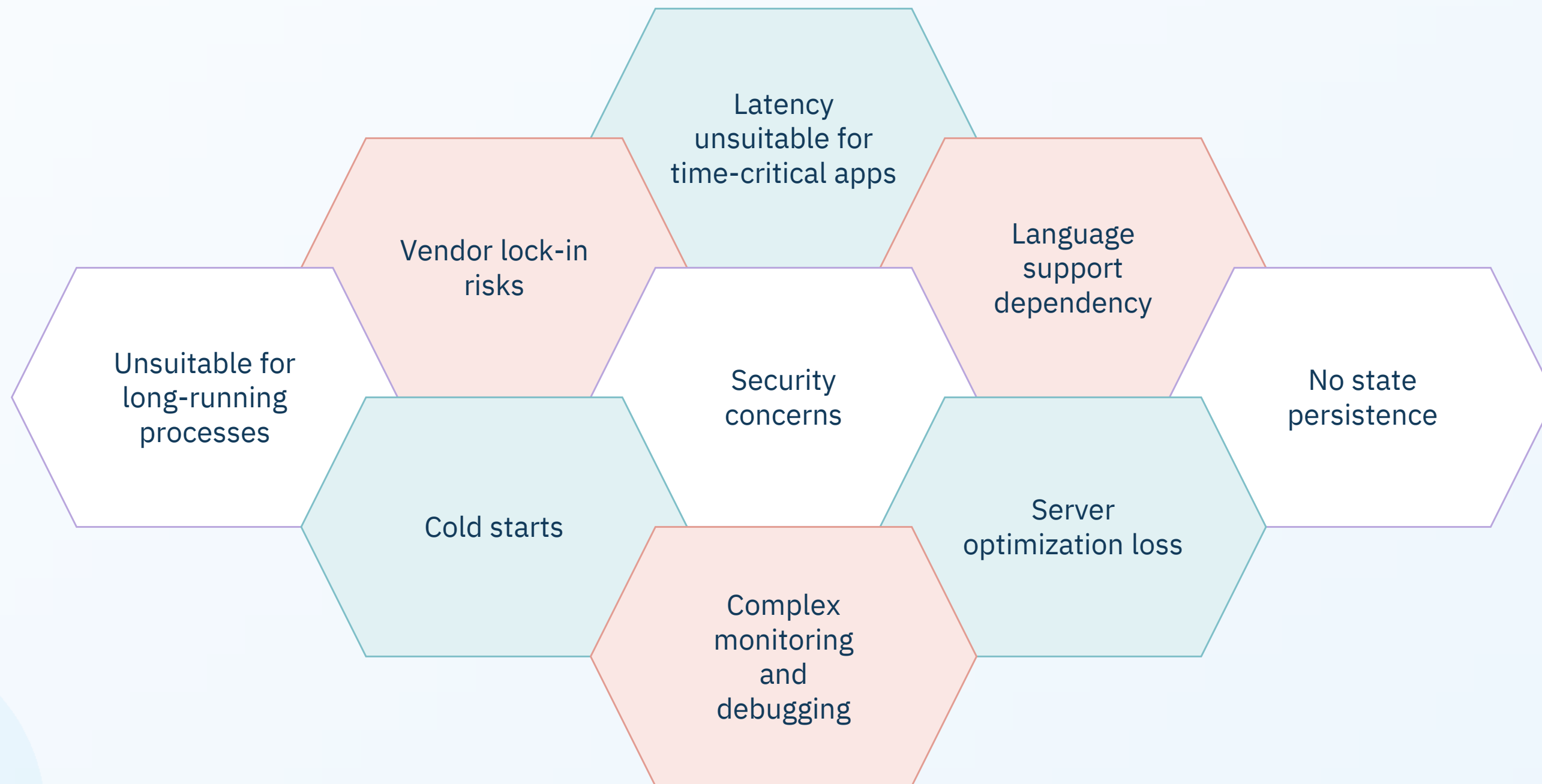
Cold start latency on the first call

A maximum execution time per function

Only the provider's supported runtimes

Managed services create vendor lock-in

Serverless computing constraints



Serverless versus containers

Build serverless first then move to container

Pros of serverless and cons of containers

Feature	Serverless Pros	Containers Cons
Cost	Low – only pay for what you use	High – pay when running or idle
Scalability	Provider manages it, scales as demand varies	Developers need to plan and take care of it
Maintenance	Cloud provider manages all infrastructure	Developers update container and OS with patches
Time to deploy	Milliseconds	Seconds
Speed to market	Development is fast so orgs can focus on core business	Setup and launch time slow down the dev process

Serverless versus containers

Pros of containers and cons of serverless

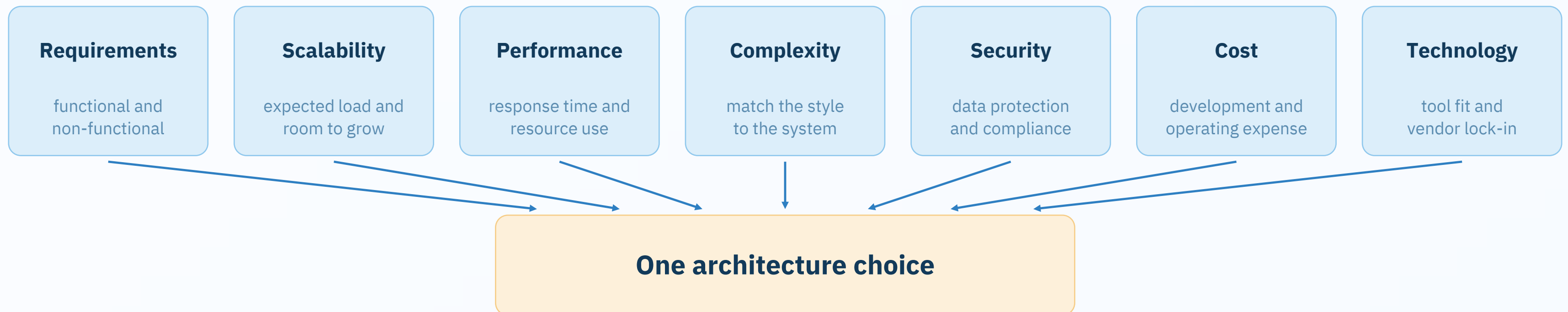
Feature	Serverless Cons	Containers Pros
Testing	Hard – backend replication locally is challenging	Easy – run the same container locally and in cloud
Portability	Hard to port across providers	Easy to port as containers are provider agnostic
Latency	Cold starts a challenge; unideal for time-critical apps	Low latency – ideal for time-critical workloads
Batch jobs	Unideal for long-running apps (function time limit)	Best for long-running apps as no time restrictions
App control	Developers can only configure CPU and memory	Can configure anything (apps and resources)
Language support	Only popular programming languages	Any language

 PART 07

Choosing an architecture

— CHOOSING AN ARCHITECTURE

Architecture selection factors



Monolithic and microservices

Monolithic

SUITS

Simple, well-understood requirements

Rapid delivery with little overhead

One team building and maintaining it

DOES NOT SUIT

High traffic or rapid growth expected

Frequent independent releases needed

Microservices

SUITS

Large system with distinct features

Parts that must scale separately

Several teams releasing independently

DOES NOT SUIT

Small applications, where overhead wins

No distributed systems experience

— CHOOSING AN ARCHITECTURE

SOA and serverless

Service-oriented architecture

SUITS

Integrating many enterprise systems

Services reused across applications

Legacy systems must take part

DOES NOT SUIT

Low-latency communication is essential

The bus overhead outweighs the benefit

Serverless

SUITS

Unpredictable or variable workloads

Pay-per-use cost is a priority

Small functions delivered quickly

DOES NOT SUIT

Long-running or compute-heavy jobs

Vendor lock-in must be avoided

— CHOOSING AN ARCHITECTURE

Client-server, peer-to-peer, event-driven and layered

Client-server

SUITS

Centralised control of data

Conventional web applications

DOES NOT SUIT

The server becomes the bottleneck

Peer-to-peer

SUITS

Decentralised sharing of data

Resilience without a central node

DOES NOT SUIT

Strict consistency is required

Event-driven

SUITS

Asynchronous or real-time work

Components that scale apart

DOES NOT SUIT

Tracing a request must stay simple

Layered (n-tier)

SUITS

Clear separation of concerns

Maintainability is the priority

DOES NOT SUIT

Latency budget is very tight

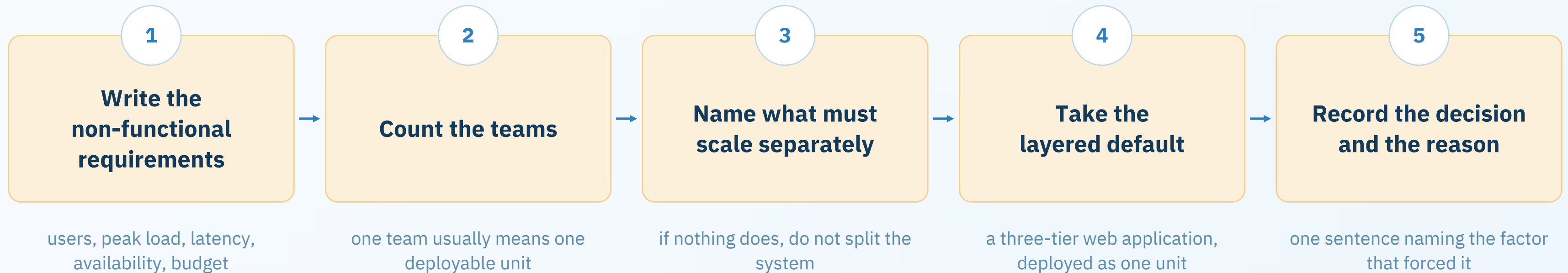
— CHOOSING AN ARCHITECTURE

Architecture selection summary

Architecture	Suits	Does not suit
Monolithic	Simple application, rapid build, one team	Growth expected, frequent independent change
Microservices	Large system, independent scaling, many teams	Small application, no distributed experience
SOA	Enterprise integration, reuse, legacy systems	Bus overhead too high, low latency essential
Serverless	Variable workload, pay per use, fast delivery	Long-running jobs, lock-in unacceptable
Client–server	Conventional web application, central control	High scalability, distributed processing
Peer-to-peer	Decentralised sharing, resilience	Complex coordination, strict consistency
Event-driven	Asynchronous work, decoupled components	Event volume unmanageable, hard debugging
Layered (n-tier)	Separation of concerns, maintainability	Performance critical, application is simple

— CHOOSING AN ARCHITECTURE

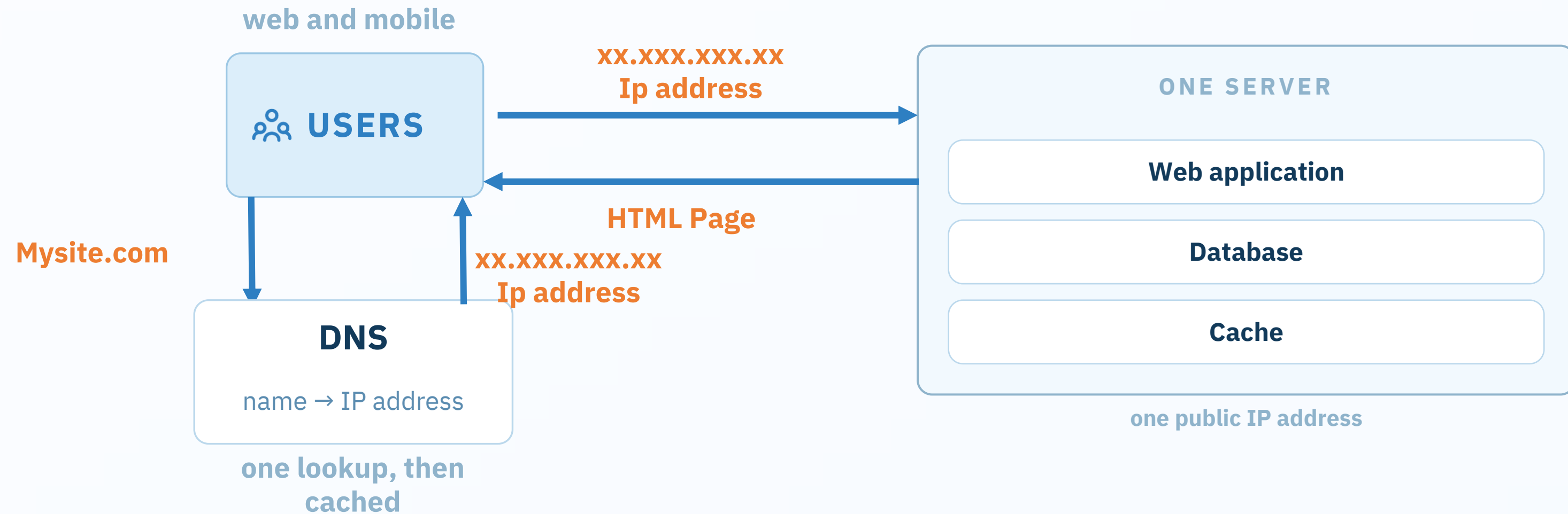
Selecting an architecture for a project



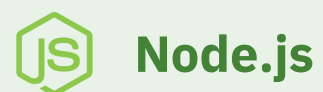
— PART 08

Scaling

Single-server deployment



A TYPICAL SINGLE-SERVER STACK



Resource contention on one server



Memory contention

The database wants memory for rows. The web wants it for connections.



CPU contention

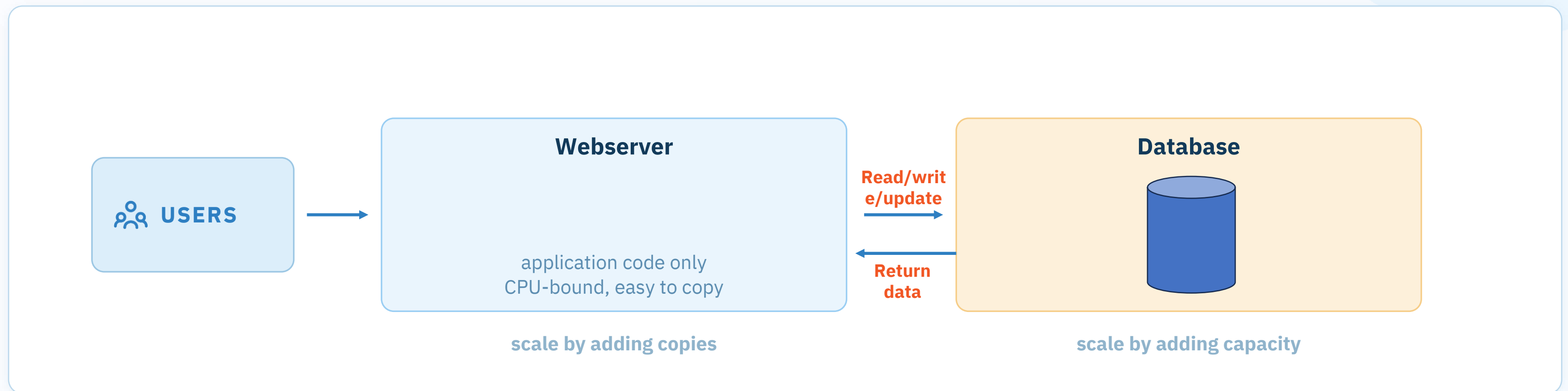
A report query pins the CPU and everything queues behind it.



Coupled resource growth

The database needs disk. The web tier needs copies. One box, one dial.

Separating the data tier



THE DATA TIER, IN PRACTICE



PostgreSQL



MySQL



Amazon RDS



MongoDB

— SCALING

Relational and non-relational databases

RELATIONAL

Tables and rows.
Joins across tables in SQL.
The default, and the right
default for most systems.

NON-RELATIONAL

Key-value, document,
column and graph stores.
Joins are generally
not supported.



PostgreSQL



MySQL



Oracle



Cassandra



DynamoDB



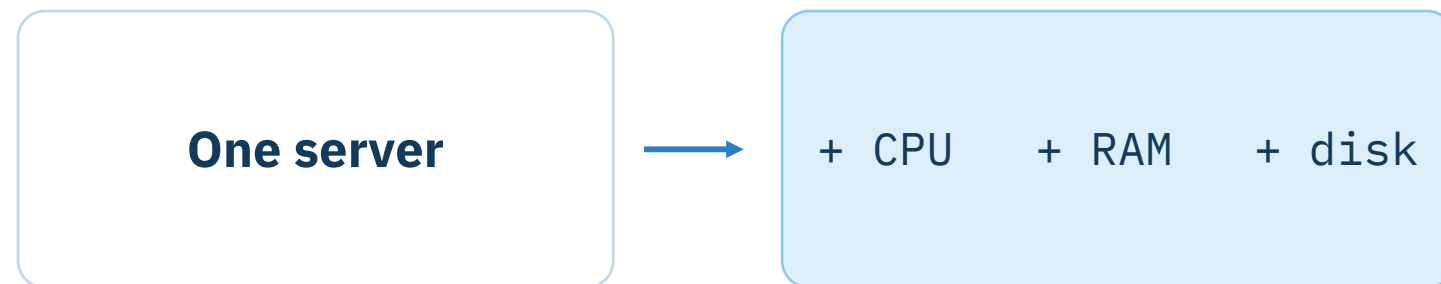
MongoDB

— SCALING

Vertical and horizontal scaling

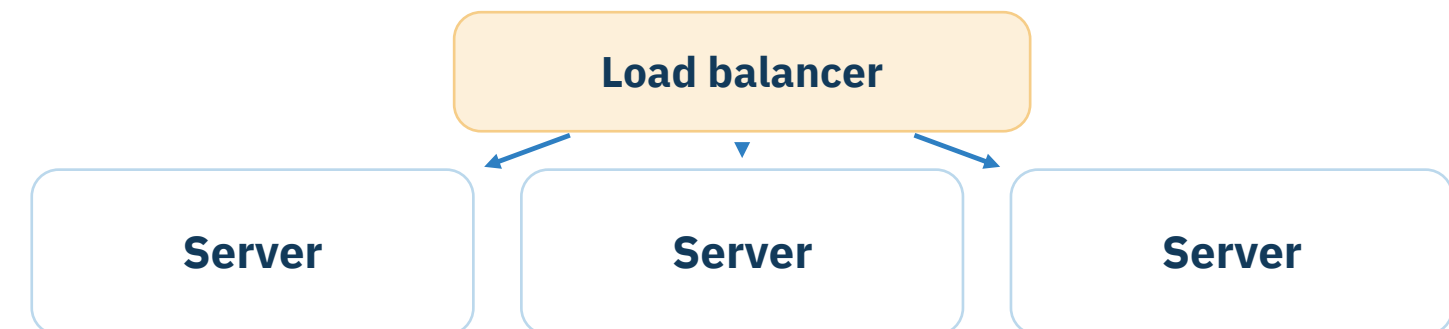
VERTICAL · SCALE UP

Add CPU, RAM and disk to the server you have.



HORIZONTAL · SCALE OUT

Add more servers to the pool.



AUTO SCALING SERVICES



AWS EC2



Kubernetes

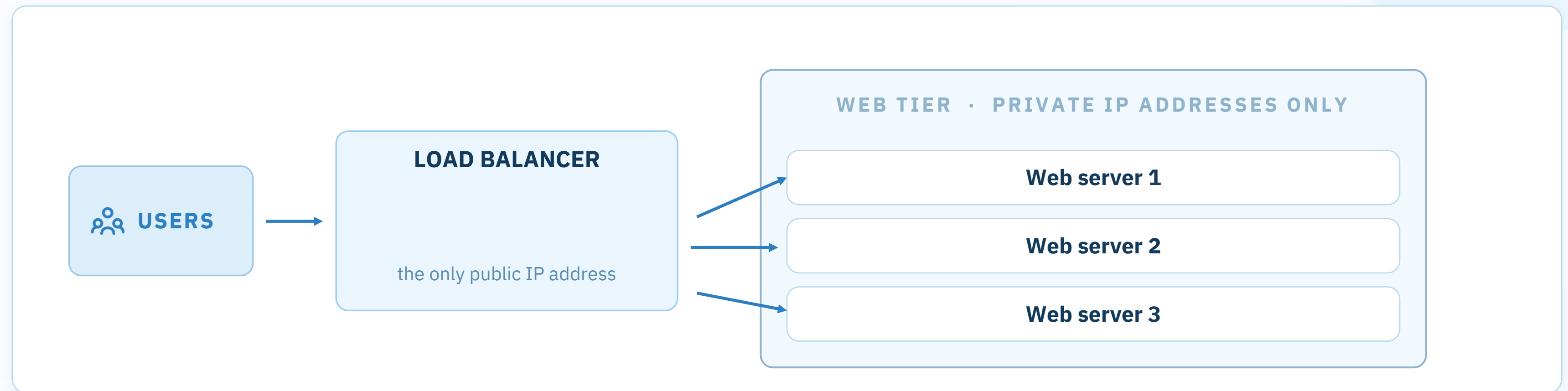


AWS Auto Scaling

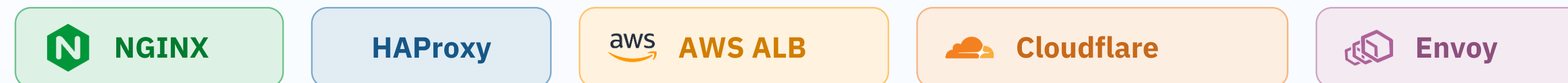


Cloud Run

Load balancer



LOAD BALANCERS IN PRODUCTION



— SCALING

Load balancer benefits



Server failure

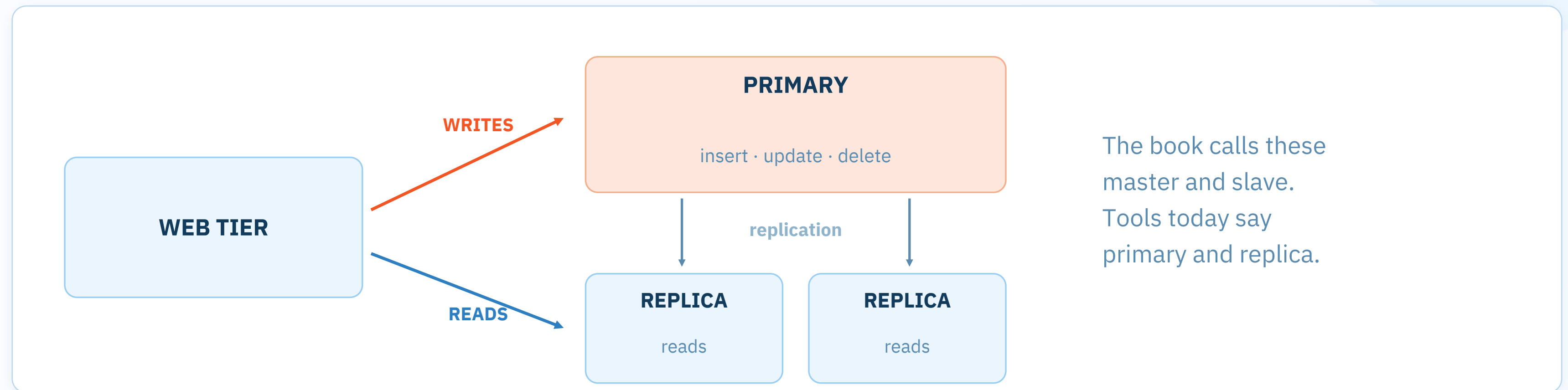
Traffic goes to the servers that answer their health check. Nobody sees an error page.



Traffic growth

Add a server to the pool and it starts receiving requests. No DNS change, no downtime.

Database replication



DATABASES WITH BUILT-IN REPLICATION



PostgreSQL



MySQL



Amazon RDS



PlanetScale

Replica failure and promotion



Replica failure

Reads move to the other replicas. If it was the only one, reads go to the primary until it is back.

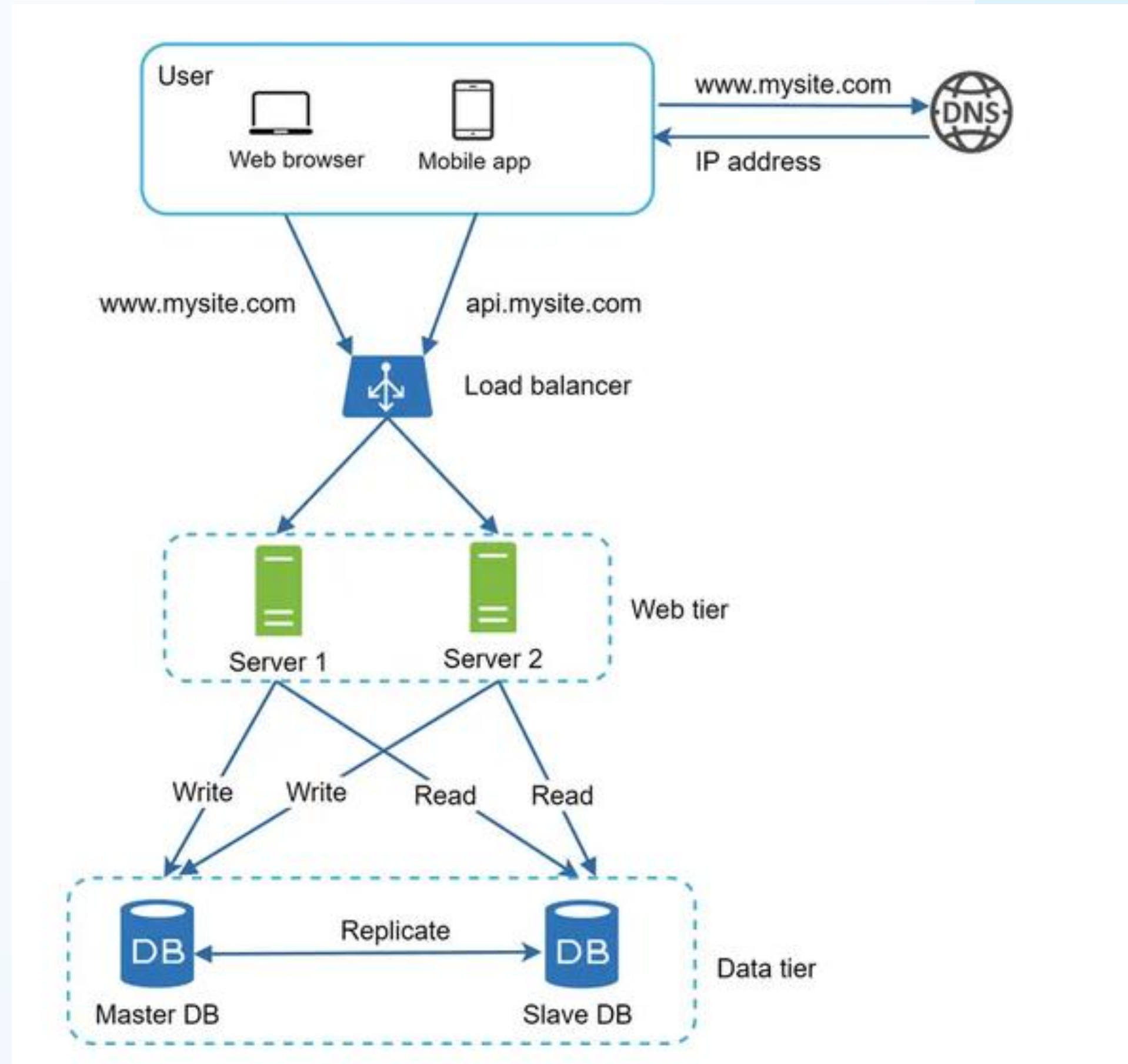


Primary failure

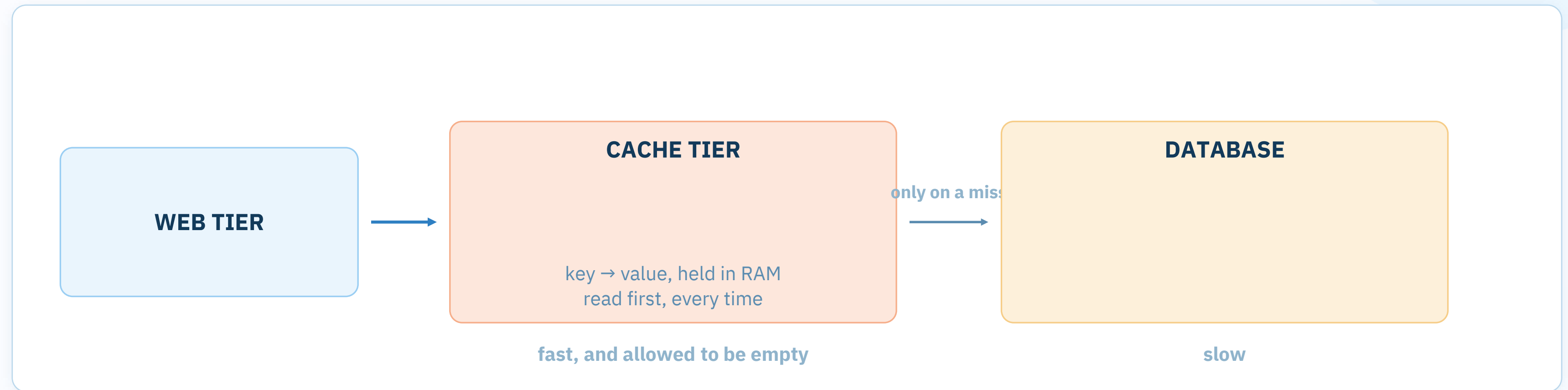
A replica is promoted to primary. Writes resume there. Recovery scripts replay whatever had not replicated yet.

— SCALING

Design so far



Cache tier



THE CACHE TIER, IN PRACTICE



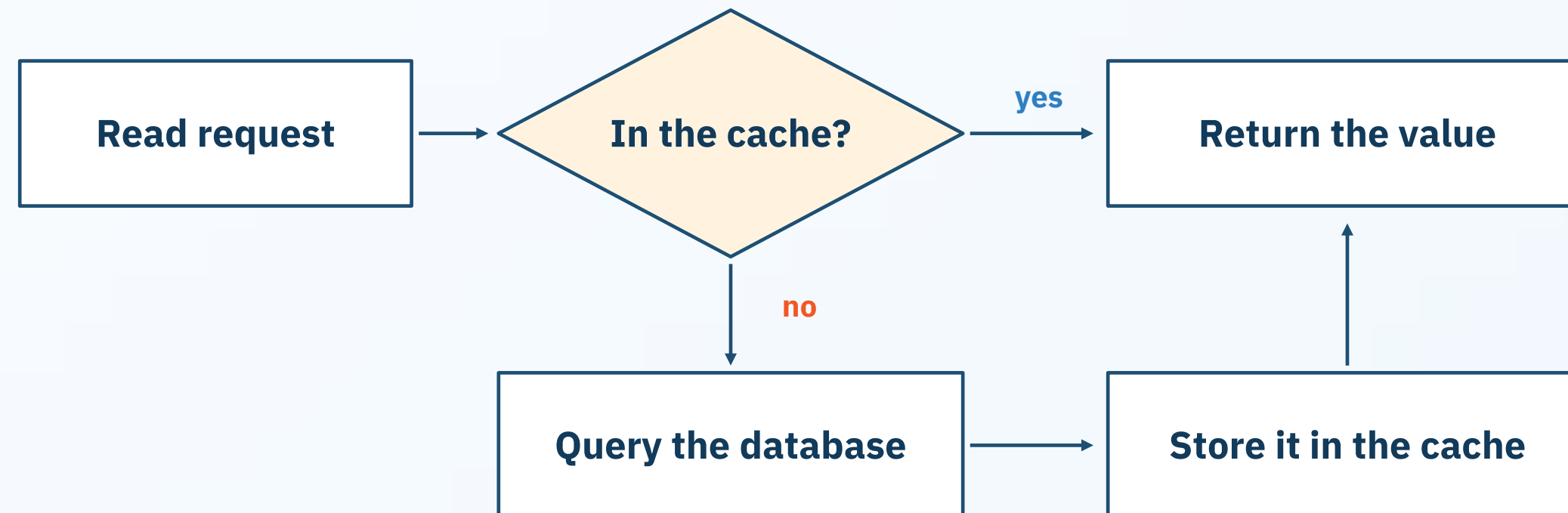
Redis

Memcached



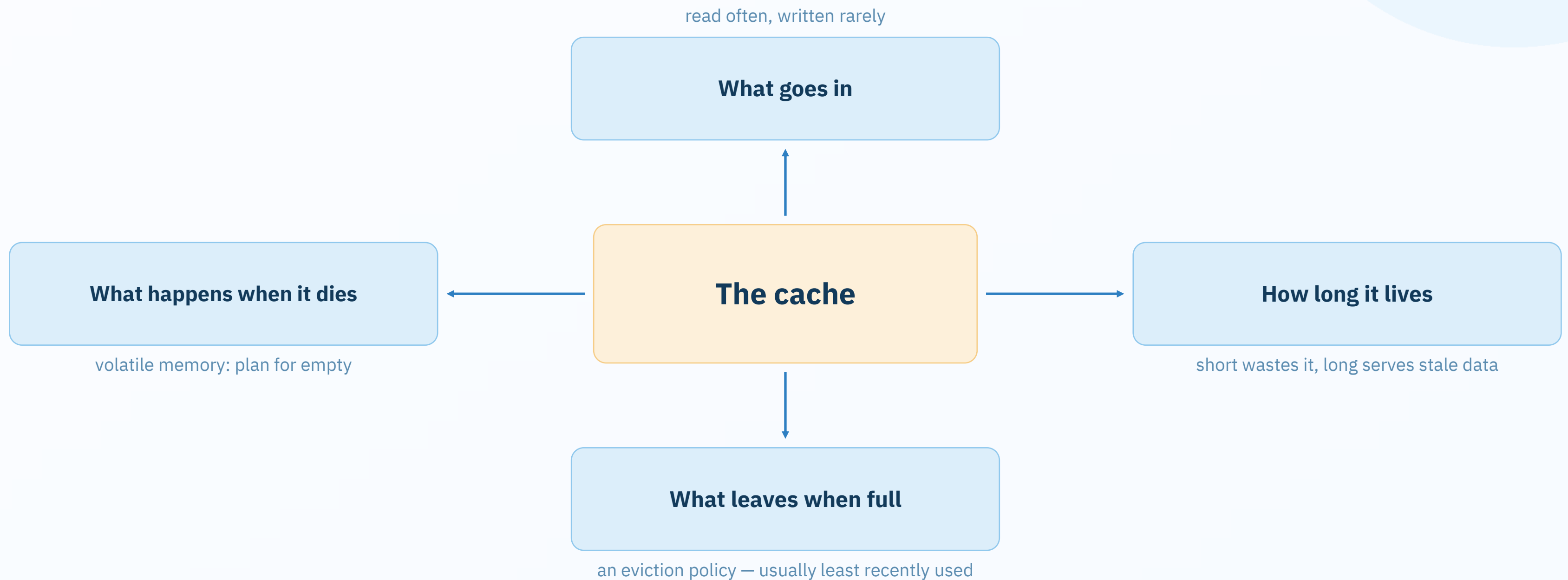
ElastiCache

Read-through cache

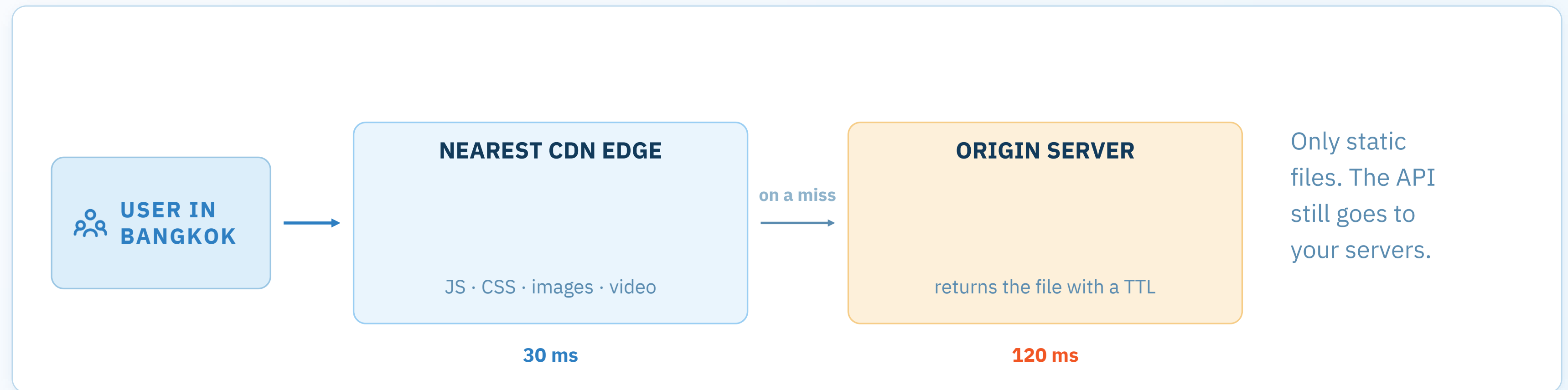


The cache is never written to directly.

Cache design decisions



Content delivery network



CONTENT DELIVERY NETWORKS

 **Cloudflare**

 **CloudFront**

 **Akamai**

 **Fastly**

CDN operating consideration



Expiry policy

Too short and the edge keeps asking the origin. Too long and users see last week's page.



Cache invalidation

Use the provider's API, or version the URL: `image.png?v=2`.



CDN fallback

IF CDN failed, the client should notice and fall back to the origin.

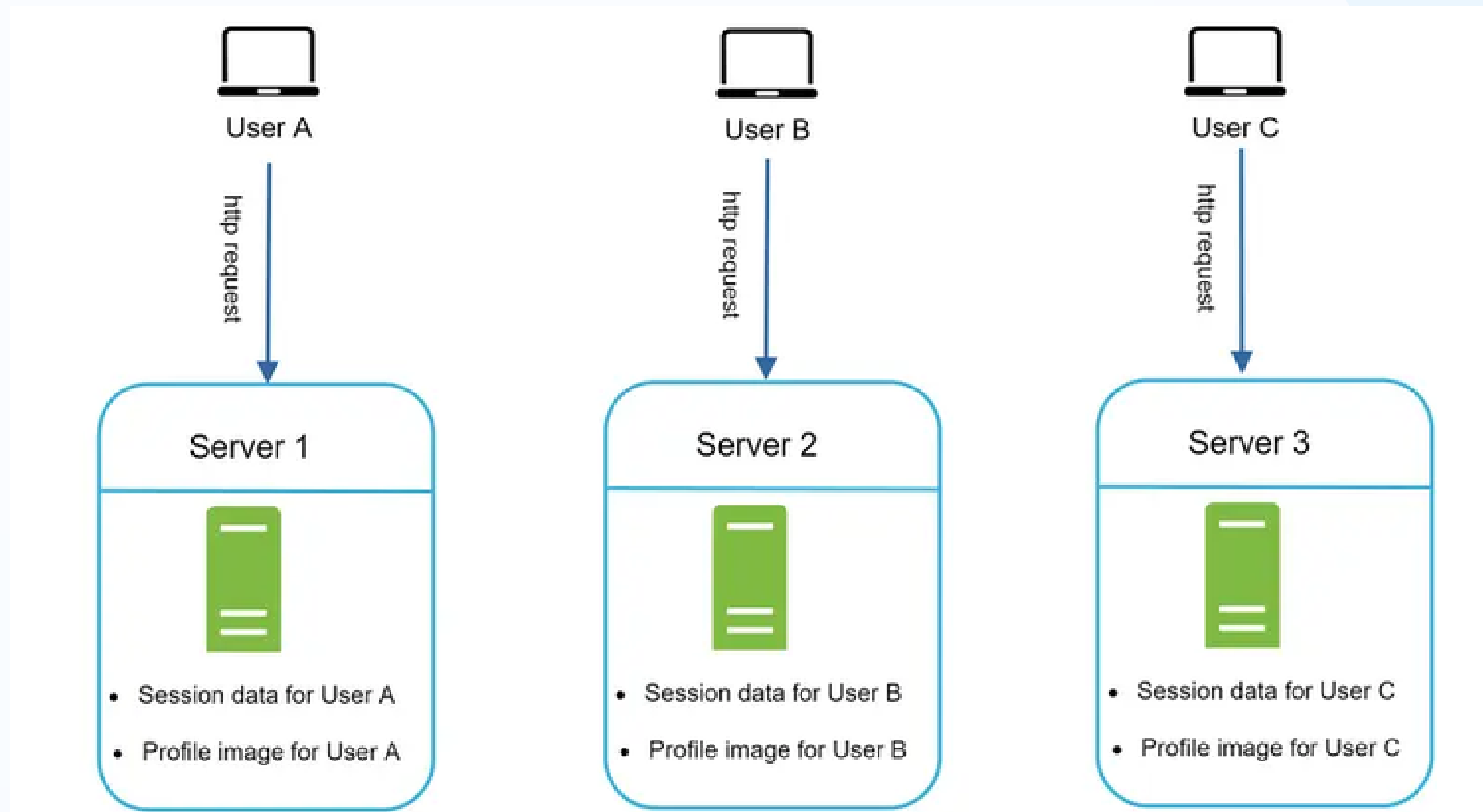


Transfer cost

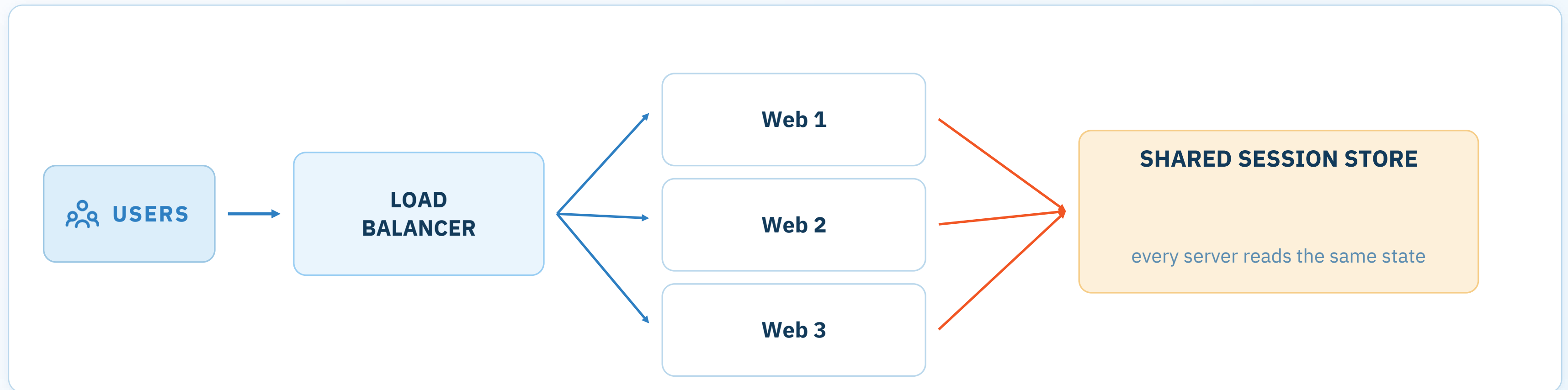
Putting rarely-read files on a CDN costs money and improve nothing.

— SCALING

Stateful web tier



Stateless web tier



SHARED SESSION STORES



Redis

Memcached



DynamoDB



JWT

— SCALING

Auto scaling



Interchangeable servers



Elastic capacity



Auto scaling

A rule on CPU or queue depth adds the servers for you.

AUTO SCALING POLICIES

 **AWS Auto Scaling**

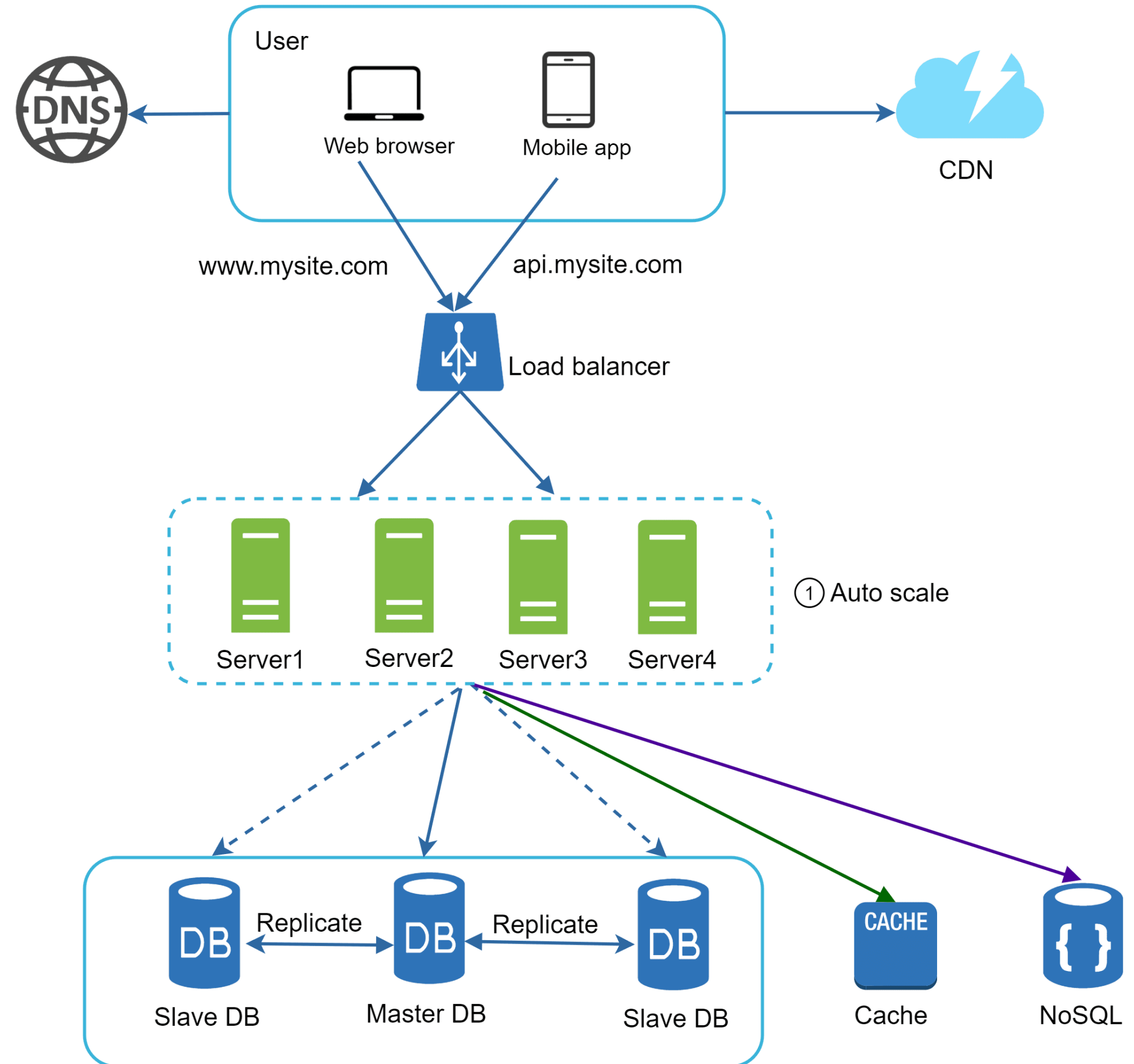
 **Kubernetes**

 **Cloud Run**

 **Docker**

— SCALING

Design so far 2



Single-region limitations



Latency from distance

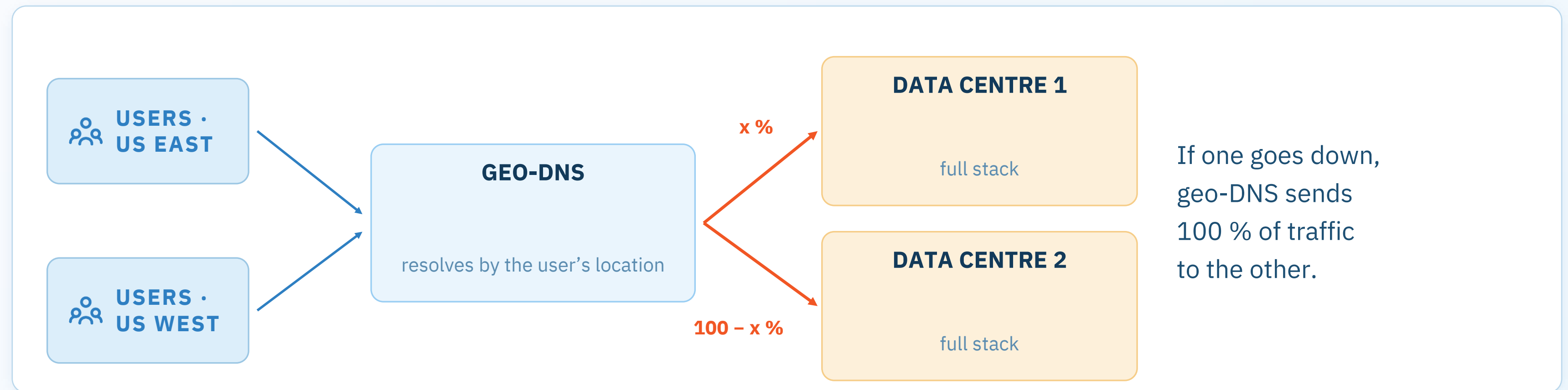
A user on the other side of the world waits for every round trip, and no amount of code fixes the speed of light.



Single point of failure

A power failure, a network partition or a provider incident takes the whole service with it.

Multiple data centres



ROUTING BY LOCATION

 **Route 53 Geo**

 **Cloudflare**

 **Google Cloud**

 **Azure**

Multi-region trade-offs



Traffic routing

Geo-DNS has to be correct, and change quickly when a region fails.



Cross-region consistency

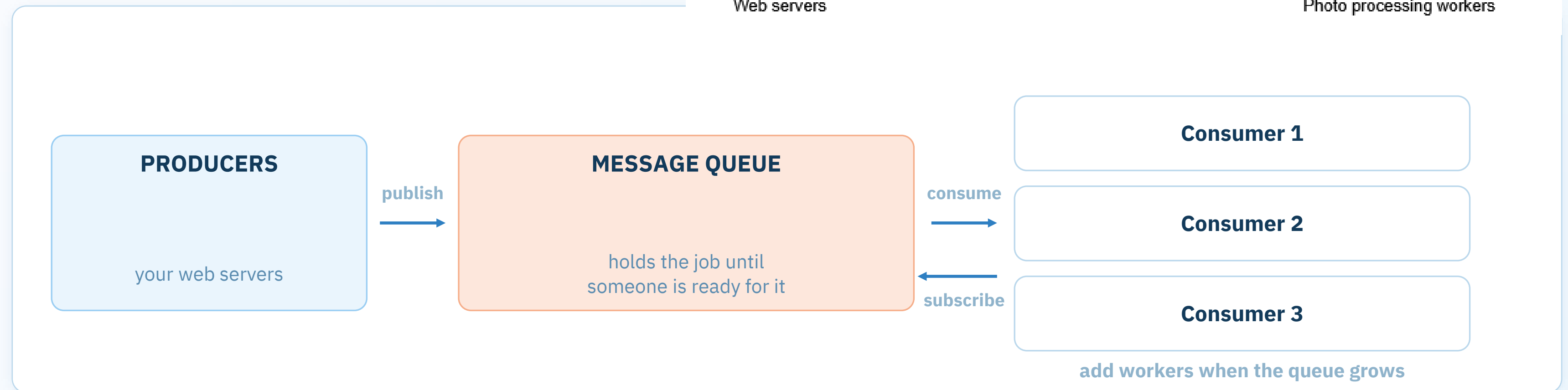
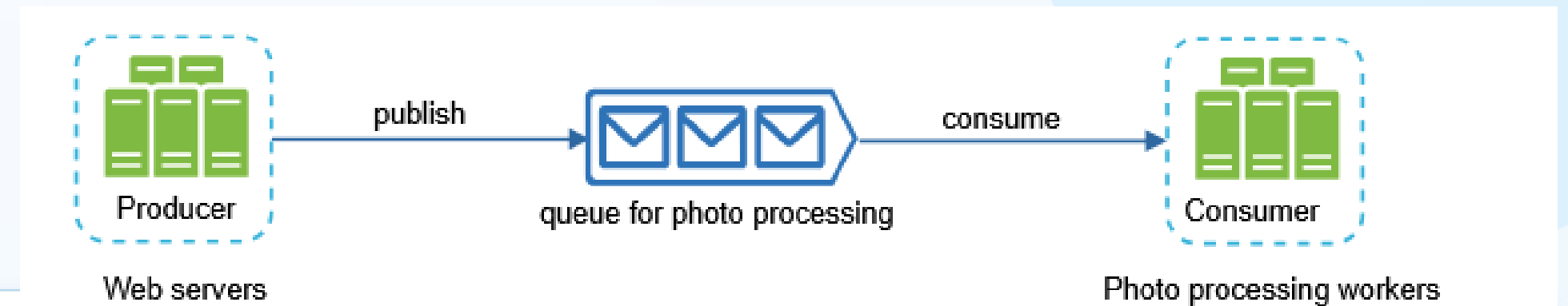
Replication across regions is asynchronous, so the copies are never equal.



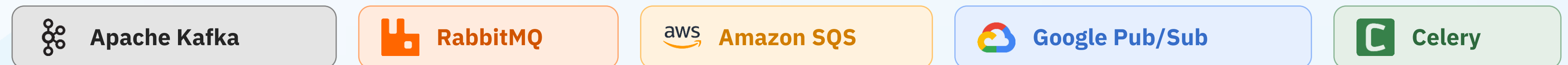
Testing across regions

Every release has to reach every region, and be tested in each.

Message queue



QUEUES AND JOB RUNNERS



— SCALING

Decoupling benefits



Asynchronous processing

The producer publishes and answers the user. The job runs later.



Worker downtime

Messages wait in the queue. Nothing is lost while workers restart.



Independent scaling

Ten web servers and two workers, or the other way round.

LIGHTWEIGHT ALTERNATIVES

BullMQ



Celery



Amazon SQS

Logging, metrics and automation



Logging

What happened, in order —
collected into one searchable place.



Metrics

Numbers over time. Enough history
to say if today is unusual.



Automation

Build, test and deploy without a
person typing the steps each time.

COMMON TOOLING



Elasticsearch



Prometheus



Grafana



Datadog



Sentry

Metric levels

Business level

daily active users, retention, revenue

Aggregated level

how the whole database tier or cache tier is doing

Host level

CPU, memory and disk I/O on each machine

Build and deployment automation



Continuous integration

The pipeline tests each commit, so a broken merge is caught in minutes.



Continuous delivery

Build, test and deploy run the same way every time.

COVERED IN WEEK 9



GitHub Actions



Jenkins



Argo CD



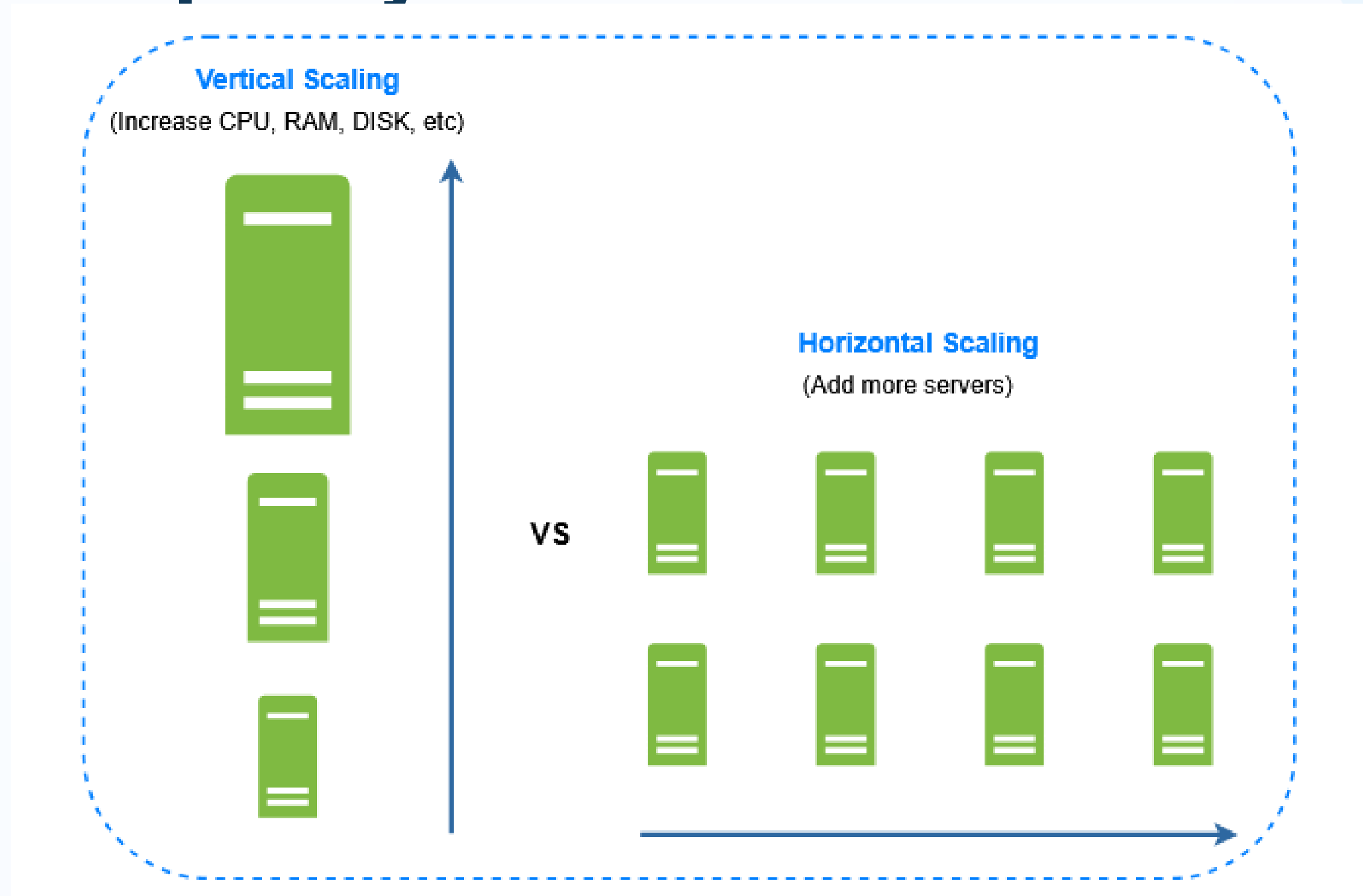
Terraform



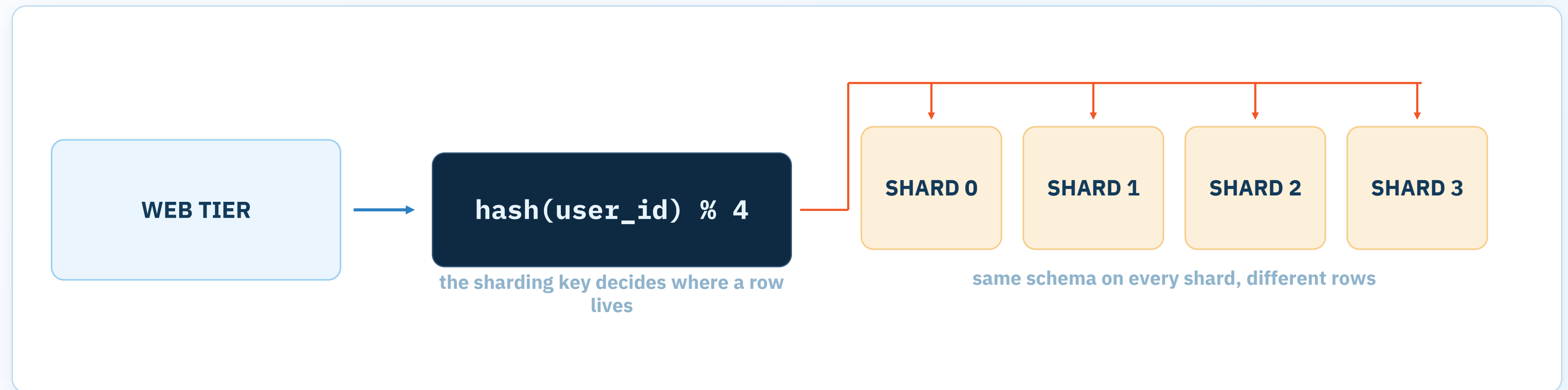
Docker

— SCALING

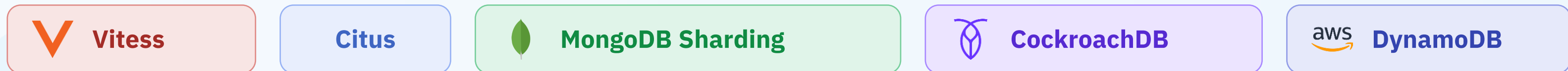
Database capacity limits



Database sharding

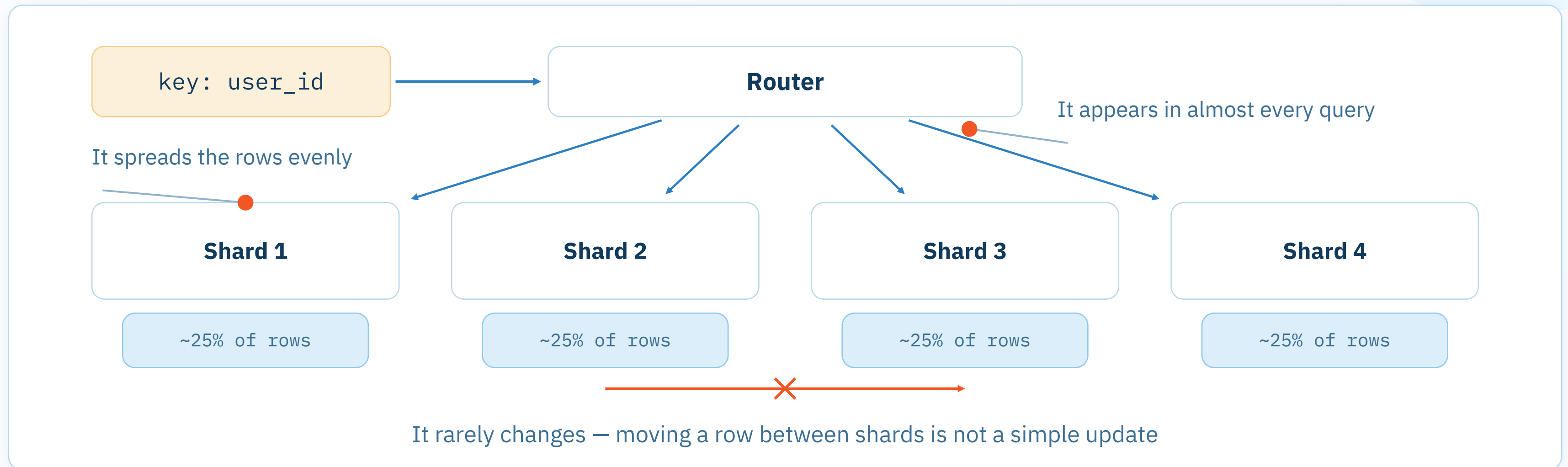


MANAGED SHARDING



— SCALING

Choosing the sharding key



Sharding trade-offs



Resharding

When one shard fills up, or fills faster than the others, the data has to be redistributed. Consistent hashing is the usual answer.



The celebrity problem

One very popular row sends all its traffic to one shard. That shard needs its own machine, or its own split.

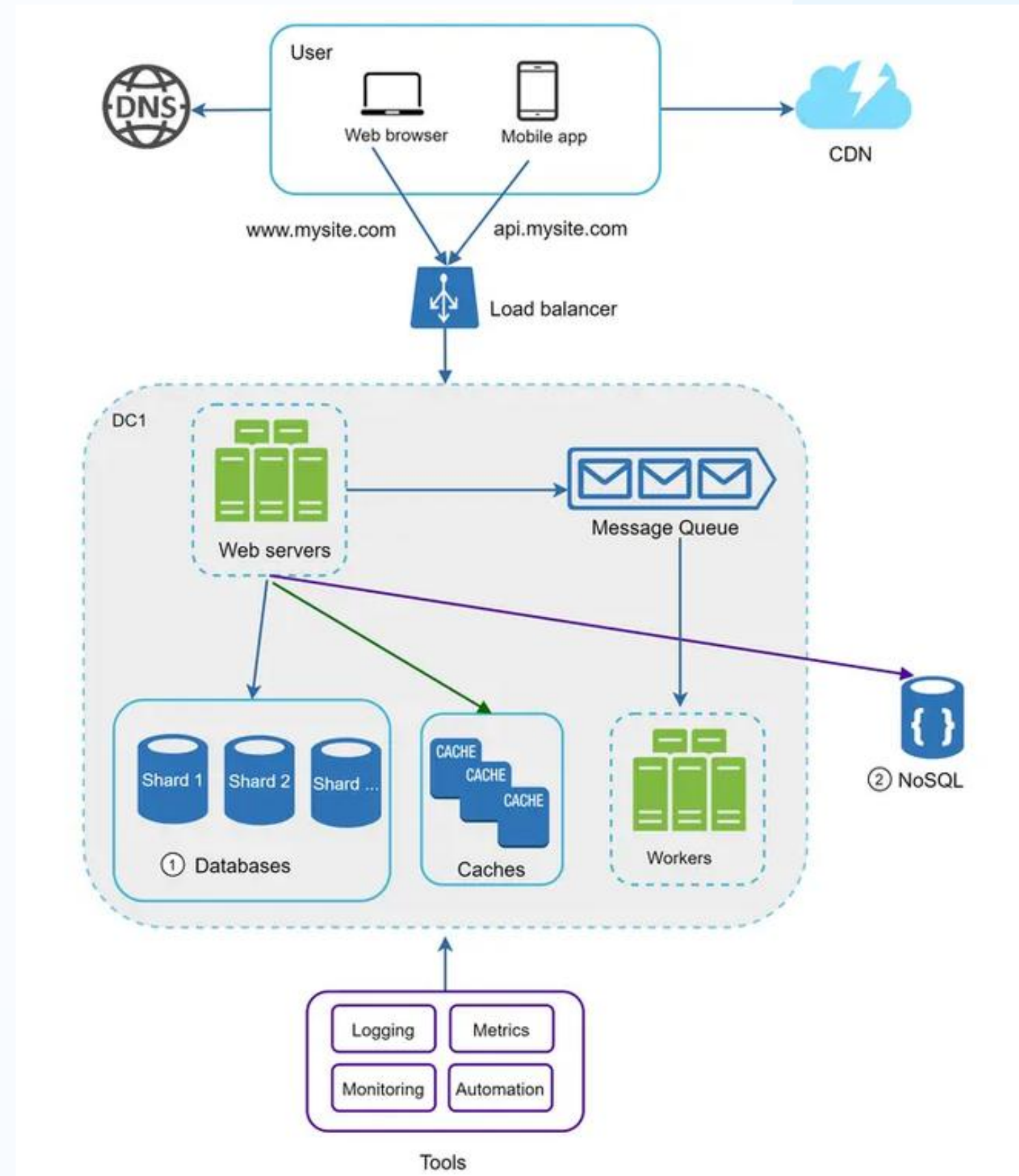


Cross-shard joins

A join across shards is impractical, so the schema is de-normalised until each query touches one table.

— SCALING

The complete design



Techniques for availability



Stateless web tier

Any server can answer any request.



Redundancy at every tier

One of anything is a single point of failure.



More than one region

For when losing a data centre is not acceptable.

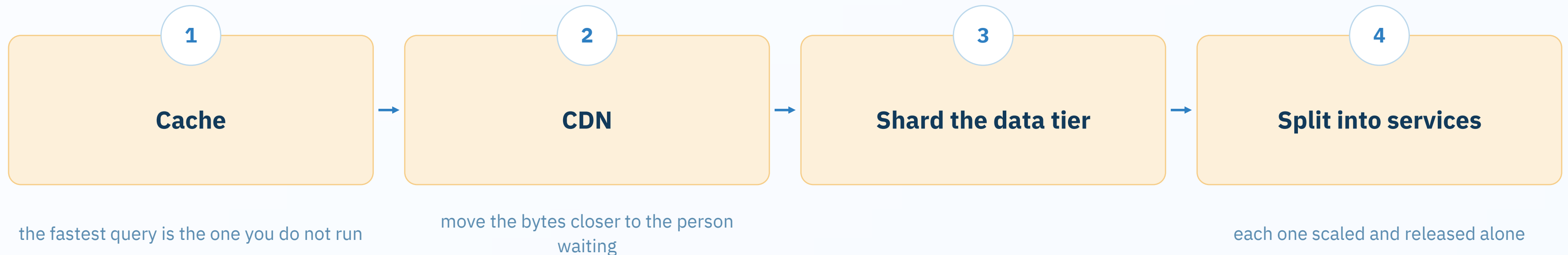


Monitoring and automation

You cannot operate what you cannot see.

— SCALING

Techniques for performance



 PART 10

Technology stacks

Four common stacks

MERN



MongoDB

ex Express.js



React

JavaScript across the whole stack

MEAN



MongoDB

ex Express.js



Angular

JavaScript with a full MVC framework

LAMP



Linux



Apache



MySQL

Open-source stack on relational data

Ruby on Rails



Ruby



Rails



PostgreSQL

One framework, convention over configuration

MERN stack components



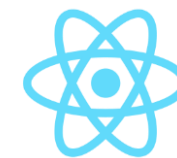
MongoDB

Document database.
Records are JSON-
shaped, with no fixed
schema.



Express.js

Web framework on
Node.js. Defines routes
and middleware.



React

Component-based
interface library with a
virtual DOM.



Node.js

Server-side JavaScript
runtime with non-
blocking I/O.

One language on the client, on the server and in the database query

MERN and MEAN compared

	MERN	MEAN
Front end	React — a component library	Angular — a full MVC framework
Data binding	One-way	Two-way
State management	Redux or the Context API	Angular services and RxJS
Learning curve	Easier, component based	Steeper, more concepts
Flexibility	Flexible, suits dynamic interfaces	More opinionated structure
Mobile	React Native	Ionic or NativeScript
DOM	Virtual DOM	Real DOM
Typical use	Modern web development	Enterprise projects

MERN and LAMP compared

	MERN	LAMP
Database	MongoDB — document	MySQL — relational
Server language	JavaScript on Node.js	PHP, Perl or Python
Web server	Node.js with Express	Apache HTTP Server
Concurrency	Non-blocking I/O	Process or thread per request
Scaling	Horizontal, asynchronous by default	Horizontal with server configuration
Maturity	Newer, fast-moving ecosystem	Long established and stable

MERN and Ruby on Rails compared

	MERN	Ruby on Rails
Language	JavaScript	Ruby
Database	MongoDB — document	Relational, usually PostgreSQL
Shape	Four tools assembled by the team	One full-stack MVC framework
Set-up effort	More configuration to write	Convention over configuration
Security	Handled per component	Built-in defaults
Best at	Interactive, real-time interfaces	Rapid delivery of standard applications

Stack selection

Stack	Suits
MERN	Single-page applications, real-time features, one JavaScript team
MEAN	Enterprise single-page applications needing a structured framework
LAMP	Conventional web applications on relational data, PHP experience
Ruby on Rails	Rapid prototyping, content management, small to medium projects

The stack the team can already debug outweighs the stack that reads best

Summary



Questions?

Week 9 — continuous integration, delivery and release discipline

INSTRUCTOR

Dr. Santawat

DEPARTMENT

Computer Engineering, KMUTT

